Yesterday 6:44 PM

You said:

how to buy microduck and ship to australia?

Worked for 9s >

ChatGPT said:

You can currently pre-order the **Microduck robot from Pollen Robotics (Hugging Face)** for **US$399**, but there's one complication: **Australia is not included in the initial shipping countries.**

Pollen Robo… +1



Microduck by Pollen Robotics

US$399

Pre-order price before taxes and shipping

- 25 cm tall, 15 motors
- Open-source SDK and reinforcement learning
- Four colours available
- First deliveries targeted before Christmas 2026

Official Pollen Robotics Store ↗

How to get it shipped to Australia

I found three possible routes:

1. **Ask Pollen Robotics for direct shipping to Perth (recommended).** Their official store currently lists US, Canada, Europe, UK, Norway, Switzerland, Japan and South Korea for the launch. Australia is planned for a later stage, but no date is confirmed.
2. **Use a parcel-forwarding service.** You could order through an eligible country such as the US, then forward it to Perth. However, you should confirm the seller accepts forwarding addresses and the courier permits shipping its lithium battery.
3. **Wait for Australian availability.** This is likely the simplest option for warranty claims and returns.

The official sales contact is **sales@pollen-robotics.com**.

Pollen Robo… +1

I would email them asking whether they can arrange an Australian pre-order, even if you pay extra shipping. If they cannot, ask whether a US forwarding address is permitted.

Important: I also found Microduck sites such as `microducks.org` and `microduck.homes`. I recommend purchasing only through `store.pollen-robotics.com`, which is linked to the actual manufacturer's website.

Pollen Robotics

Would you like me to investigate **which parcel-forwarding service can deliver Microduck from the US to Perth, including estimated shipping costs and battery restrictions**?

You said:

yep

Worked for 41s >

ChatGPT said:

I checked the official Microduck store and several parcel-forwarding services that ship from the United States to Perth.

My recommendation is MyUS or Shipito, provided they confirm acceptance of Microduck's removable battery. Both offer US receiving addresses and international forwarding, but neither has specifically confirmed that it will ship this robot to Australia.

The key complication is that Microduck uses a **removable NP-F550 rechargeable battery**. A battery installed inside equipment may be treated differently from a removable battery packed separately.

Pollen Robo… +2

1. Parcel-forwarding options



MyUS — My first choice

Ships internationally and has a dedicated lithium-battery handling process. Its published fees include US$8 for batteries installed in equipment, while some dangerous-goods shipments incur a minimum US$29.99 fee.

Battery rules ↗ · Official website ↗

MyUS



Shipito — Strong alternative

Has a dedicated Australia service. It accepts eligible lithium-battery electronics, but its rules distinguish batteries installed in a device from loose or spare batteries.

Australia shipping ↗ · Restricted items ↗

Free US Ad… +1



Stackry — Backup option

Ships to Australia and handles eligible devices with installed lithium batteries. Loose and spare batteries require additional review.

Official website ↗ · Battery rules ↗

Stackry +1

2. Estimated total cost to Perth

Microduck itself weighs 780 g, but the shipping carton dimensions and packed weight have not been published. The following is a **planning estimate, not a live shipping quote**.

Cost component	Estimated USD
Microduck	$399
US domestic delivery and tax	$0–$40
Forwarding to Perth	$45–$100
Battery handling	$8–$35
Australian GST, if applicable	$40–$55
Estimated total	$492–$629

For context, MyUS publishes an example rate of US$15.93 for a small, one-pound Oceania shipment, while Stackry publishes a US$62.72 example for two consolidated phones. Neither is an actual Microduck quote.

MyUS +1

Australian GST may be collected by the seller or forwarder. Goods valued at A$1,000 or less generally do not attract import duty at the border, but that does not mean they are exempt from GST.

Australian B… +1

3. What I'd do before ordering

1. Contact Pollen Robotics about direct Australian delivery and whether they accept US forwarding addresses.

2. Ask MyUS to confirm shipping eligibility for a robot with a removable NP-F550 battery.
3. Get the packaged weight and dimensions from Pollen Robotics, then calculate a proper shipping quote.
4. Order only once both parties confirm the arrangement.

The manufacturer's current shipping policy says North American Microduck orders ship from its US warehouse, so using a US forwarder is geographically feasible. However, **I could not verify that Pollen's checkout accepts forwarder addresses**, nor that a forwarder will accept this exact battery configuration.

Pollen Robotics…

4. Email ready to send

Microduck Pre-order – Shipping to Australia

Send

sales@pollen-robotics.com

Microduck Pre-order – Shipping to Australia

Hi Pollen Robotics Team,

I'm interested in pre-ordering a Microduck for personal use in Perth, Western Australia.

I understand Australia isn't currently included in the initial launch regions. Could you please confirm:

1. Whether direct shipping to Australia can be arranged, or when Australian orders may become available.
2. Whether I can place an order using a US parcel-forwarding address (such as MyUS or Shipito).
3. The estimated packaged weight and dimensions, and whether the NP-F550 battery is shipped installed in the robot or separately.

I'd be happy to cover additional shipping costs if direct delivery to Australia is possible.

Thanks for your help.

Best regards,
Kun

You can also check the Stackry Australia shipping calculator ↗ once Pollen provides the carton dimensions.

My preference: Try direct delivery first. If that fails, use MyUS after written confirmation on the battery. This avoids committing approximately US$400 to a pre-order that might reach a US warehouse but cannot legally or practically be forwarded to Perth.

You said:

这套 radxa zero 3w 和 rasperry pi 以及 jetson orio nano 有什么区别

Worked for 45s ›

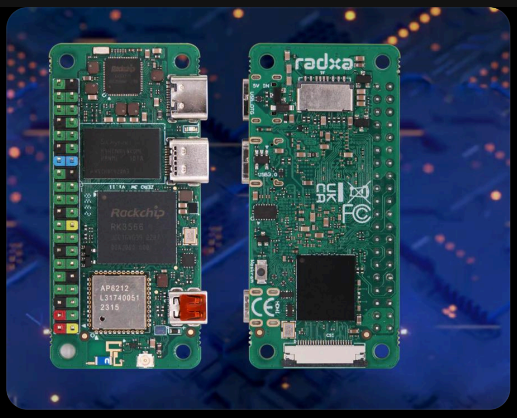
ChatGPT said:

这三种设备虽然都属于 **Single Board Computer**（**SBC**，单板计算机），但实际上定位差别很大。

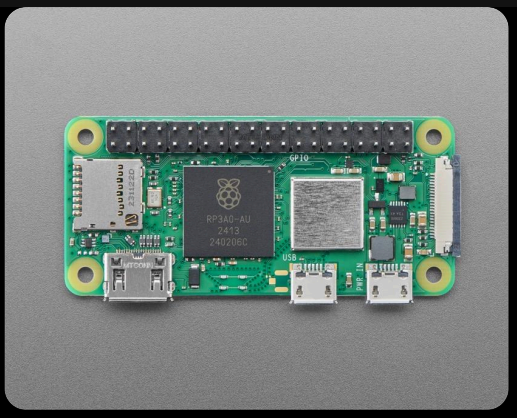
尤其结合你刚才关注的 Microduck 机器人，以及你已经拥有的 Jetson Orin Nano，我觉得最值得理解的是：

Radxa Zero 3W 是低功耗嵌入式控制计算机，**Raspberry Pi** 是通用嵌入式计算平台，而 **Jetson Orin Nano** 是专门面向 **AI** 和机器视觉的边缘计算平台。

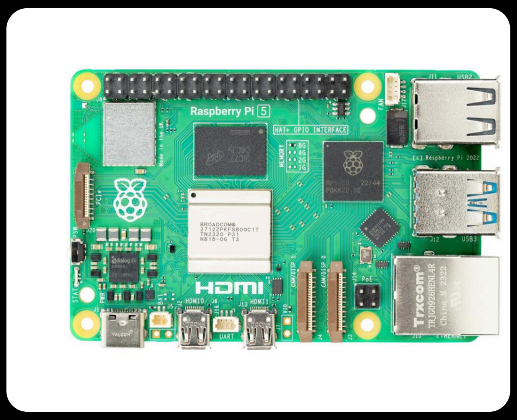
1. 先看这几块板子



Radxa Zero 3W
超小型、低功耗、适合嵌入式控制



Raspberry Pi Zero 2 W
尺寸相近、生态成熟



Raspberry Pi 5
通用 Linux 计算与开发



Jetson Orin Nano
GPU 加速、计算机视觉、本地 AI

2. 核心硬件比较

因为 Raspberry Pi 有多个版本，我选 Zero 2 W 和 Pi 5 分别作为小型与高性能代表。

指标	Radxa Zero 3W	Pi Zero 2 W	Pi 5	Jetson Orin Nano 8GB
CPU	4× Cortex-A55 1.6GHz	4× A53 1GHz	4× A76 2.4GHz	6× A78AE
GPU	Mali-G52	VideoCore IV	VideoCore VII	NVIDIA Ampere
内存	1–8GB	512MB	1–16GB	8GB
AI 加速	无专用 NPU	无	无内置 NPU	最高 67 TOPS
Wi-Fi	Wi-Fi 6	Wi-Fi 4	Wi-Fi 5	开发板无内置 Wi-Fi
存储	microSD / 可选 eMMC	microSD	microSD / 外接 NVMe	microSD / NVMe
典型供电	5V/2A	5V	5V/5A	7–25W 功率模式
主板尺寸	65×30mm	65×30mm	85×56mm	模组 70×45mm

资料：Radxa Docs +3。Jetson 的 67 TOPS 是 Super 模式下的稀疏 INT8 理论峰值，并非所有 AI 模型都能达到的实际性能。

从 CPU 的架构和频率判断，一般通用计算任务大致可以理解为：

通用 CPU 能力（定性比较）　　非基准测试结果



注意，Jetson 的 CPU 并非一定在每个单线程任务上都领先 Pi 5；它真正突出的地方是 GPU 与 Tensor Core。

3. AI 能力才是最大的区别

这里有个值得补充的细节：**Radxa Zero 3W** 的 **RK3566** 实际上也有一个约 **0.8 TOPS** 的 **NPU**，只是软件支持和可用模型受到限制，并不像 Jetson 那样拥有成熟的 CUDA/TensorRT AI 生态。

IT之家 +1

AI 应用	Radxa Zero 3W	Pi 5	Jetson Orin Nano
调用云端 LLM API	很合适	很合适	很合适
本地小型 ML 推理	可以	可以	很强
摄像头目标识别	小模型可行	可行	很强
本地语音识别	受限	小模型可行	较强
本地运行 3B LLM	不推荐	较慢	可以尝试
本地运行 7B LLM	不实际	受内存与速度限制	量化后可尝试
机器人运动控制	很合适	合适	合适，但可能过重

这些是工程适用性判断，不是统一模型下的推理速度测试。

4. 那为什么 Microduck 会选择 Radxa Zero 3W？

从公开的 Microduck 硬件拆解项目来看，Radxa Zero 3W 被用于机器人主控，并通过串口与电机控制系统通信。

GitHub

原因很可能是它的架构符合小型机器人的实际需求：



最重要的认知是：机器人的智能决策、运动策略推理和电机实时控制，可以由不同的计算单元分别承担。

例如，Microduck 不需要在机身内运行一个完整的大语言模型，才能完成行走、保持平衡、执行预训练的运动策略。

5. 如果是你来开发机器人，我会怎么选？

考虑到你已经有一台 Jetson Orin Nano，我反而建议把它保留为 AI 计算节点，不一定安装在机器人身体里。



机器人机载端：**Radxa Zero 3W**

运动策略、传感器输入、通信、指令转发。小尺寸和低功耗很适合移动机器人。

↑↓ Wi-Fi / 网络通信



独立 AI 节点：**Jetson Orin Nano**

本地语音识别、视觉感知、LLM 推理、机器人高级行为规划。

这样能够把 AI 的算力消耗、功耗和散热从小型机器人的机身分离出去。不过，平衡和安全关键的控制闭环应该保持在机载端，不能依赖有延迟和丢包风险的 Wi-Fi。

我的结论：

- 想做 Microduck 这样的微型机器人：**Radxa Zero 3W** 很合适。
- 想做一般 Linux/IoT/自动化项目：**Raspberry Pi 5** 更方便，开发生态更成熟。
- 想研究本地 AI、Vision-Language Models、机器人智能：**Jetson Orin Nano** 的价值最大。

还有一个我觉得更值得讨论的问题：

既然 **Microduck** 采用 **Radxa Zero 3W** 这样相对有限的计算平台，却能完成复杂的双足运动，那么它的运动智能到底来自强化学习训练出的 **Policy**，还是传统运动学控制算法，或者两者结合？

这其实比单纯比较板子的算力更有研究价值，也直接关系到能否自己开发一个类似 Microduck 的机器人。

You said:

我没太懂 到底什么是microduck的强化学习呢 还有他的policy呢 强化学习 我一致以为是llm训练的活， radxa zero 3w怎么干的了呢 ？是不是说policy是一套训练产出 并不需要耗太高算力？

ChatGPT said:

对！你的理解基本正确，而且抓住了 **Microduck** 强化学习架构中最关键的一点：**Policy** 是强化学习训练的产物，而机器人在实际运行时，只需要执行这个已经训练好的 **Policy**，并不需要重新训练。

你之前把强化学习（Reinforcement Learning，RL）主要理解成 LLM 训练的一部分，其实它是一个独立而广泛的机器学习范式，在机器人控制领域尤其重要。

我们从架构角度把这件事拆开。

1. 强化学习不等于 LLM 训练

强化学习的基本过程是：

1. Agent（智能体）观察环境当前的状态。
2. 根据状态选择一个 Action（动作）。
3. 环境执行动作，进入新的状态。
4. Agent 获得 Reward（奖励）或 Penalty（惩罚）。
5. 重复大量试验，学习什么动作最有利于长期目标。

对于 LLM，Action 可以是生成下一个 Token。

对于 Microduck 这样的机器人，Action 则可能是多个关节电机的目标位置。

例如：

当前状态 **State**	动作 **Action**	奖励 **Reward**
身体向左倾斜	调整腿部关节	避免跌倒 +1
向前走了一步	继续调整步态	前进 +2
摔倒了	—	-10
保持平衡	微调关节	稳定 +1

这些数值只是解释强化学习机制的示例，并非 Microduck 官方的实际奖励函数。

经过成千上万甚至数百万次仿真试验，智能体可能逐渐学会稳定站立、左右转弯、前进等行为。

2. Policy 到底是什么？

Policy（策略）本质上是一个从观察到动作的映射函数。

写成数学形式：

$$a_t = \pi_\theta(o_t)$$

其中：

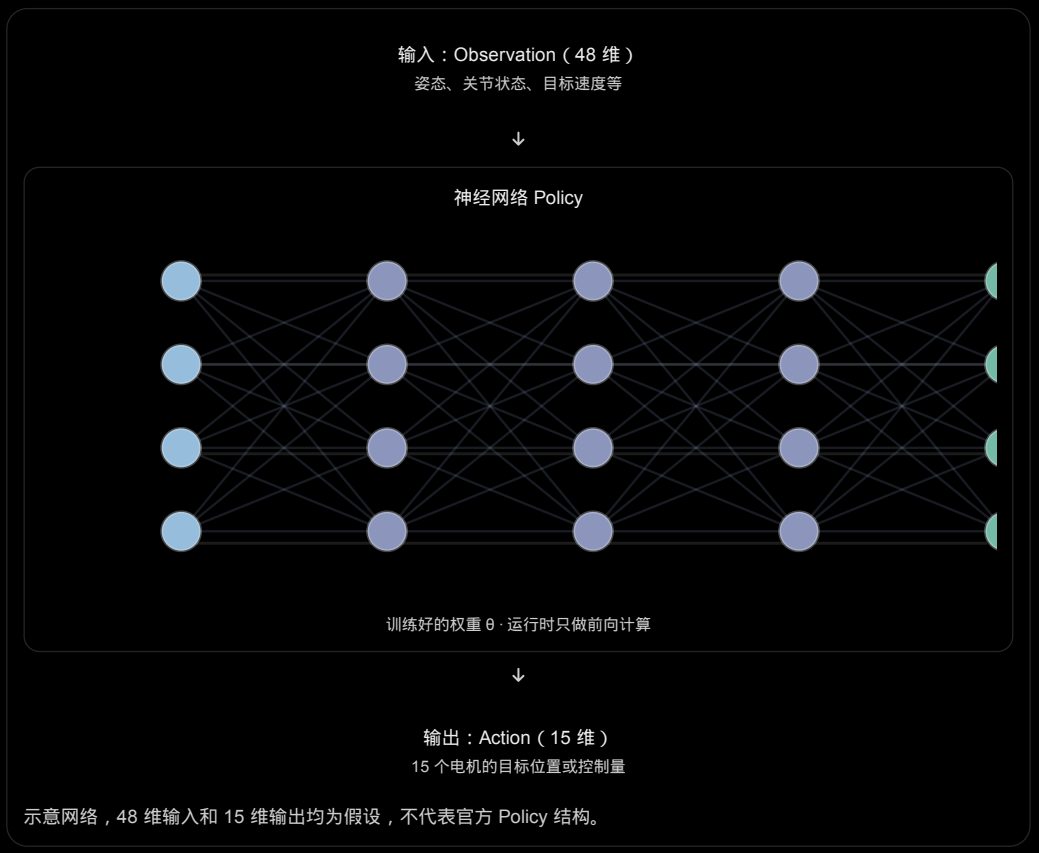
- o_t：当前观察值，例如 IMU 姿态、关节位置、关节速度及目标行走速度。
- π：Policy，即策略函数。
- θ：训练得到的参数（例如神经网络权重）。
- a_t：输出的动作，例如各个电机的目标角度。

它的含义非常直接：

> 给我机器人的当前状态和运动目标，我就告诉你下一步各个关节应该如何运动。

这个 Policy 可以实现为一个很小的神经网络。

例如，一个假设的机器人 Policy：



示意网络，48 维输入和 15 维输出均为假设，不代表官方 Policy 结构。

这里最有意思的是，**Policy** 并不一定知道什么是走路，也不必内置复杂的人类编写的步态规则。

它通过训练学到一种行为规律：当前处于这样的状态，采取什么动作，长期效果会更好。

3. Microduck 实际是怎么训练和运行的？

我刚查了 Pollen Robotics 官方网站及其 GitHub，确认它采用的是：

- **MuJoCo**：物理仿真环境。
- **PPO（Proximal Policy Optimization）**：强化学习算法。
- **ONNX**：训练完成后的模型导出格式。
- **50 Hz**：机器人本地运行 Policy 的控制频率。

Pollen Robo… +1

整个流程是：

阶段 **A**：训练（**Training**） 高算力

MuJoCo 物理仿真



这就是为什么 Radxa Zero 3W 能承担任务：它根本不需要做 **PPO** 训练，只需要执行一个相对小的神经网络的前向推理。

而且即使没有强大的 GPU，CPU 也完全有可能在 20ms 内完成这样的小模型推理。是否达标取决于模型参数量、推理框架、调度和其他任务的负载。

4. 那么 Policy 的推理到底有多轻量？

假设我们设计一个神经网络：

$$48 \to 128 \to 128 \to 15$$

其权重参数量约为：

$$48 \times 128 + 128 \times 128 + 128 \times 15 = 24,448$$

加上 Bias，总共约 **24,719** 个参数。

FP32 权重占用约 97 KB。

这仅是演示规模，并非 Microduck 官方网络的实际参数数量。

对比一下：

模型	参数规模
上面的示例运动 Policy	约 25K
小型语言模型	约 100M–1B
Llama 3 8B	8B
大型 LLM	数百亿乃至更多

所以在这个示例中，一个控制 Policy 的参数量可以比 8B LLM 小几十万倍。

这不是说所有机器人 Policy 都很小，而是说明完成复杂的物理运动，并不必然需要庞大的模型。

5. 还有一个更重要的问题：训练好的 Policy 是不是固定动作？

不是。这是 Policy 和传统动画式动作控制的本质区别。

传统预设动作：

`左腿抬起 → 左腿落下 → 右腿抬起 → 右腿落下`

即使机器人被推了一下，它也可能继续按原本的动作序列执行。

强化学习 Policy：

`读取实时状态 → 推理适合当前状态的动作 → 执行 → 再观察`

也就是说，模型参数是固定的，但输出动作是动态变化的。

因此即使没有继续学习，Policy 仍然可以对环境变化作出反应。至于它能应对多大的扰动，取决于训练时见过哪些状态、随机化范围及实际控制性能。

6. 你现有的 **Jetson Orin Nano** 在这里能做什么？

这就有意思了。

你完全可以研究这样一条技术路径：

- 用 PC 或云端 GPU，运行 MuJoCo + PPO 来训练 Microduck 的运动 Policy。
- 导出 ONNX Policy。
- 用 Radxa Zero 3W 执行低延迟运动控制。
- 用 Jetson Orin Nano 运行摄像头视觉、语音识别、LLM 或更复杂的高级决策模型。

这样，Jetson 负责决定机器人应该做什么，Radxa 负责根据实时状态决定关节如何运动。

但要注意，Jetson Orin Nano 虽然有 AI GPU，也不代表适合高效运行所有基于 GPU 并行仿真的强化学习训练流程；训练框架与硬件兼容性仍需单独确认。

官方技术资料可以直接看：

- Microduck 官方介绍 ↗
- Microduck 机器人运行代码 ↗
- Microduck 强化学习训练代码 ↗

最后还有一个重要的机器学习概念： Policy 不一定是神经网络，也可以是规则、表格或概率分布。在 Microduck 这里，它是训练得到的神经网络策略。

你现在可以把它理解成：

$$\boxed{\text{RL Training} \longrightarrow \text{Policy Model} \longrightarrow \text{Real-time Inference}}$$

这和 LLM 的 Pre-training → Model Weights → Inference 在系统架构层面非常相似，只不过学习目标、训练算法和模型规模不同。

而我认为下一步最值得深入的是：**PPO** 究竟如何让机器人自己学会走路？它如何从最初的随机电机动作，逐渐训练出稳定步态？ 这个过程可以直接用你正在学习的机器学习知识来理解。

You said:

我从头跟一遍 microduck的组装 调试 训练 能把这个经历 润色成简历一部分 方便日后找个正式的机器人相关工作么

Worked for 1m 15s ›

ChatGPT said:

完全可以，而且我认为这是一条很适合你切入机器人行业的路径。

但我建议不要把这件事仅仅定位成“购买 Microduck、组装机器人、学习强化学习”，而是把它做成一个可以证明你具备机器人软件研发能力的正式工程项目。

结合你已有的 20 多年软件架构、云平台、系统集成和 DevOps 经验，以及正在攻读的 Data Science 硕士，你最有竞争力的转型方向不是初级 **Robotics Engineer**，而是 **Robotics Software Engineer**、**Robotics Platform Engineer**，进一步可以定位到 **Robotics Software Architect** 或 **Physical AI Engineer**。

这里的关键是把过去的软件工程能力与新掌握的机器人技术结合起来，而不是从零开始与机器人专业毕业生竞争。

一、不同投入深度，对简历的价值完全不同

你完成的工作	对机器人职位的证明力
购买、组装、调试 Microduck	较低：主要证明动手能力
部署官方软件、运行预训练 Policy	基础：机器人 Linux 环境、运行与故障诊断
自己修改 Reward，重新训练 PPO Policy	较高：具备实际强化学习工程经验
在真实机器人部署、优化自己的 Policy	高：具备 Sim-to-Real 实践经验
实现新的行为，量化性能改进	很高：具备独立研发能力
向官方仓库贡献被合并的 PR	很高：形成可公开验证的工程成果

尤其是后面三项，才会让招聘经理真正感兴趣。

二、为什么 **Microduck** 值得作为作品集项目？

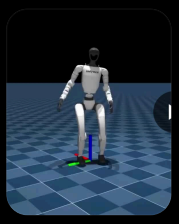
我查看了官方源码仓库，发现它并不只是一个玩具机器人项目，而是一个相当完整的小型机器人软件系统。



Microduck Runtime ↗

使用 Rust 实现，包含 50 Hz 电机控制循环、15 个舵机、通信、设备配置、更新与回滚，以及摄像头和传感器服务。

GitHub



Microduck RL Training ↗

使用 MuJoCo、mjlab、PPO、Domain Randomization 和 Sim-to-Real，导出 ONNX Policy 部署到机器人。

GitHub

这意味着你可以围绕一个实际平台，同时积累两条重要的技术经验：**Robotics Software Engineering** 与 **Robot Learning**。

而且官方提供模拟器，即使机器人尚未运到澳大利亚，你也可以先开始研究。

Pollen Robotics

三、我建议你将项目设计为五个阶段

01 硬件集成与控制系统调试

完成组装或硬件配置、传感器校准、电机测试、Linux 环境部署和基础运动控制。

交付：架构图、硬件配置文档、故障分析与调试记录。

02 强化学习与运动控制

复现官方 PPO 训练，理解 Observation、Action、Reward、Episode、Policy Network，修改奖励函数并训练自己的步态。

交付：训练代码、Reward 曲线、不同 Policy 的对比实验。

03 **Sim-to-Real** 部署和验证

将自己的 ONNX Policy 部署到实体机器人，对比仿真与真实环境的表现，并针对摩擦、机械间隙、电机响应差异进行调优。

交付：实机演示、稳定性测试、延迟和性能数据。

04 机器人平台工程与系统集成

增加 ROS 2 接口或 Bridge，实现 Telemetry、远程指令、Policy 版本管理、自动化部署与回滚。

交付：独立开发的软件模块、GitHub Repo、测试与发布 Pipeline。

05 **Physical AI** 与自主行为

利用 Jetson Orin Nano 增加视觉识别、环境感知和高级任务决策，例如识别目标并自主接近。

交付：完整的 Perception → Planning → Motion Control 系统演示。

这五个阶段里，前三个建立机器人专业能力，第四个发挥你现有的软件架构优势，第五个则可以把你带到 Embodied AI / Physical AI 方向。

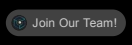
需要留意一点：Microduck 官方软件是 Rust 架构，并非天然基于 ROS 2。所以第四阶段要自己实现接口，不能只照着教程配置。

四、澳洲机器人招聘市场是否真的看重这些技能？

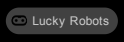
看重，而且可以找到与你的背景较匹配的方向。

我查到的澳洲招聘信息中：

- Intelligent Robotics 将 C++、Python、ROS 2、Perception、SLAM、Physical AI、测试验证列为其技术方向。

- Lucky Robots 在墨尔本招聘机器人仿真引擎工程师，涉及实时系统与物理仿真。

- RUMATA 的岗位涉及 Simulation、Embodied AI、Policy Learning 和 Whole-body Control。

- Perth 也有海事自主系统相关岗位，要求 C++、Python、ROS 2，并涉及从仿真到真实环境测试。

这里有一个需要客观看待的差距：机器人核心研发岗位通常还要求运动学、动力学、控制理论和 ROS 2 的实际使用能力。仅仅会训练 PPO，尚不足以完全覆盖这些要求。

因此，我建议你优先考虑 **Robotics Software / Platform / Edge AI** 相关岗位，再逐步向机器人运动控制算法或 Research Engineer 发展。

五、完成后，简历应该怎么写？

这段项目放在简历的 **Selected Engineering Projects** 或 **Robotics & AI Projects** 下面，而不是冒充企业正式工作经历。

下面是一份完成上述项目后可以使用的英文样稿。它是*目标版本*，只有当对应工作真实完成、有证据可以展示时才可以写入正式简历。

Writing

Robotics & Embodied AI Engineering — Independent Project

Microduck Bipedal Robot | Reinforcement Learning, Sim-to-Real & Edge AI

Technology: Python, Rust, MuJoCo, PPO, ONNX, Embedded Linux, ROS 2, NVIDIA Jetson, GitHub Actions

- Developed and validated reinforcement learning-based locomotion policies for a 15-actuator bipedal robot using MuJoCo simulation and Proximal Policy Optimization (PPO).
- Implemented and evaluated custom reward functions, domain randomization techniques, and motion-control configurations to improve walking stability and robustness.
- Deployed trained neural-network policies through ONNX to embedded robotic hardware and evaluated inference performance within a 50 Hz control loop.
- Investigated simulation-to-reality discrepancies and refined control strategies using physical test results, actuator feedback, and motion telemetry.
- Developed robotics integration components connecting onboard motion control with higher-level perception and decision-making services.
- Established version-controlled model deployment, automated validation, telemetry collection, and rollback processes for robotic software releases.

Project Evidence: Source code, reproducible training configurations, simulation results, deployment documentation, benchmark reports, and real-robot demonstrations.

这份项目经历的价值取决于你能否在技术面试时详细解释并演示其中的工作，尤其是你独立修改了什么、为什么修改、结果改善了多少。

例如，“把摔倒率降低了 30%”就比“熟悉 PPO 算法”更有说服力——但前提是你实际完成了有明确测试条件和基线的对比实验。

六、我最建议你做的三个关键成果

成果 **A**：自己训练一个明显不同于官方的 **Policy** Robot Learning

例如让 Microduck 在受到轻微推力后恢复平衡，并对比原始 Policy 的成功率。

这是证明你理解 RL，而非只是运行别人代码的重要证据。

成果 **B**：实现完整的机器人软件交付链 Platform Engineering

从训练、模型版本管理、仿真验证、实机部署，到监控和回滚。

这一部分可以充分利用你的 DevOps 和软件架构经验。

成果 **C**：让机器人具有基本自主行为 Physical AI

例如使用摄像头检测目标，由 Jetson 处理视觉和任务决策，再向 Microduck 的本地运动控制器发送目标速度指令。

我还建议整个项目都采用可复现的工程实践：每个实验保留 Git Commit、模型版本、超参数、训练曲线、测试条件和视频。这样不仅有作品集，还具备研究报告甚至技术论文的素材。

另外，考虑到 Microduck 目前是预售状态，你不必等实体设备到手才开始。可以立即从官方 MuJoCo 模拟环境和 PPO 训练仓库入手。硬件部分则应以实际收到的产品及可维护性为准，不要假设购买的成品一定需要从零组装。

七、我的职业定位建议

以你的资历，我不建议为了进入机器人行业，直接放弃已有的高级软件工程和架构定位。

比较合理的过渡路径是：

Senior Software / Solution Architect

↓

Microduck + RL + ROS 2 + Sim-to-Real

↓

Robotics Software / Platform Engineer

↓

Robotics Software Architect / Physical AI Engineer

如果以未来 6–12 个月为期限，持续完成有质量的独立开发、实机测试和开源贡献，这个项目有机会成为你转型机器人软件领域的有力证明。但它不能自动替代多年生产级机器人项目经验，直接竞争高级机器人控制或研究职位仍有挑战。

我认为你最值得追求的，不是“我会用 **Microduck**”，而是“我能够设计、训练、验证并交付一个完整的机器人智能控制系统”。

这两个描述在招聘市场上的分量完全不同。

而且你可以把这个工程项目与 Data Science 硕士学习结合起来，用实际机器人研究强化学习、最优化、概率模型和控制系统。这会比单纯跟着教程学习更系统，也更有长期职业价值。

Yesterday 10:50 PM

You said:

什么是 radxa emmc?

ChatGPT said:

Radxa eMMC 是 **Radxa** 单板计算机使用的一种板载存储芯片（**Embedded MultiMediaCard**），主要用来安装操作系统、保存程序和数据。

你可以把它理解为：类似焊接在主板上的 **SSD**，但采用不同的存储接口和技术，通常比 **SSD** 慢，却比普通 **microSD** 卡更适合长期运行嵌入式系统。

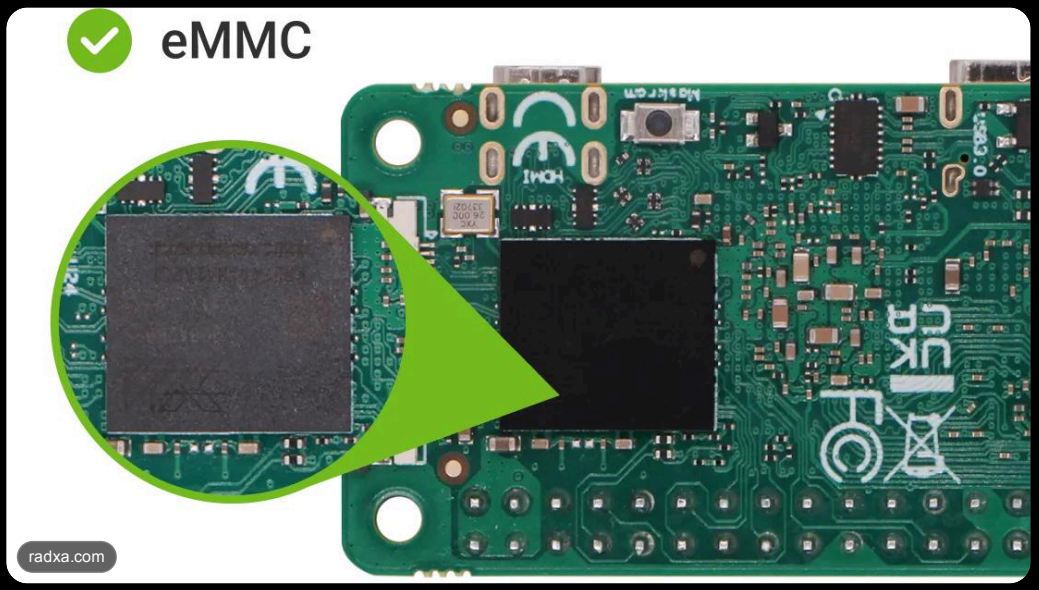
对于我们刚才讨论的 **Radxa Zero 3W** 和 **Microduck**，eMMC 的作用就是保存 Linux 系统、机器人控制软件以及训练好的 Policy 模型。

eMMC、microSD 和 SSD 有什么区别？

对比	eMMC	microSD	NVMe SSD
类型	嵌入式闪存	可拆卸存储卡	高性能固态硬盘
性能	通常较快且稳定	依卡片等级而异	最快
可靠性	通常适合嵌入式设备	消费级卡差异较大	通常较高
更换	通常不能直接更换	很容易	视设备设计而定
Linux 系统安装	支持	支持	支持，需硬件兼容
适合 Microduck	很合适	适合开发测试	通常没必要

这里要注意，eMMC 的优势不是绝对的。一张高端 microSD 卡的表现可能超过某些低端 eMMC。

Radxa Zero 3W 的 eMMC 是可选的



根据 Radxa 官方规格，它有以下版本：

Radxa Docs

- **0GB eMMC**：没有板载 eMMC，需要通过 microSD 等方式提供系统存储。
- **8GB / 16GB eMMC**：可以安装精简 Linux 系统和基本软件。
- **32GB / 64GB eMMC**：有更充分的空间存放程序、日志和机器人 Policy。

还要注意两个很容易混淆的参数：

- **RAM 4GB**：运行程序时使用的内存，断电后数据消失。
- **eMMC 32GB**：永久存储空间，保存 Linux、软件和模型文件。

例如，`Radxa Zero 3W 4GB RAM + 32GB eMMC` 就是具有 4GB 运行内存和 32GB 板载存储的设备。

针对你的 Microduck 项目

如果你准备长期做机器人调试、Policy 部署和软件开发，我建议选择 **4GB RAM + 32GB eMMC** 或更高配置，前提是它与 Microduck 的硬件、系统镜像和安装方式兼容。

eMMC 不是 AI 加速器。它不会直接提升 Policy 的神经网络推理算力，但能帮助机器人更稳定地保存和加载操作系统、控制程序及模型。

还有一点：**Radxa Zero 3W** 的 **eMMC** 是板载的，不是普通可以随手插拔的 **eMMC** 卡，因此最好在购买时就决定容量。

Radxa Download

如果你是在 Microduck 的购买页面看到 eMMC 选项，可以把具体配置发给我，我可以帮你判断是否值得加钱。

You said:

那 radxa zero 3w 怎么还有没有 emmc的版本？

ChatGPT said:

因为 **Radxa Zero 3W** 不一定需要 **eMMC** 才能运行 **Linux**。

它还有一个 **microSD** 卡槽，因此可以通过两种方式存储和启动操作系统：

1. 有 **eMMC** 的版本：可以把 Linux 系统直接安装在板载 eMMC 中。
2. 没有 **eMMC** 的版本：插入一张写好 Linux 系统镜像的 microSD 卡，同样可以启动和运行。

所以，0GB eMMC 并不意味着没有存储能力，而是没有预装板载存储，需要自己提供 **microSD** 卡。

为什么厂商还要卖无 **eMMC** 版本？

主要有三个原因：

- 降低成本：不需要额外购买板载存储芯片。
- 方便开发测试：可以准备几张 microSD 卡，分别安装不同版本的 Linux，直接换卡测试。
- 灵活选择容量：可以使用 32GB、64GB 或更大的 microSD 卡，不受固定 eMMC 容量限制。

Radxa 官方也明确支持使用 microSD 作为 ZERO 3W 的启动介质。

Radxa Docs +1

对你的 **Microduck** 项目，如何选择？

配置	我的建议
无 eMMC + 32GB microSD	适合入门、学习与调试
16GB eMMC	足够运行较精简的机器人系统
32GB eMMC	更推荐长期部署与反复测试
64GB eMMC	适合需要大量日志或本地数据的情况

但有一个重要区别：

无 **eMMC** 的 **Zero 3W** 并非低性能 **CPU** 版本。 如果 CPU、RAM 配置相同，它与带 eMMC 的版本在计算能力上基本相同，差异主要在存储速度、容量及可靠性。

所以，如果你是为了学习 Microduck 的强化学习和 Policy 部署，我会倾向于 **4GB RAM + 32GB eMMC**。不过如果你准备购买的是官方 Microduck 套件，最好先确认它所需的板子版本与系统镜像是否兼容，不要直接按一般 SBC 的选购逻辑替换配置。

Yesterday 11:58 PM

You said:

https://www.xgorobot.com/duck.html#preorder

他这里面提到的 mjlab 是什么

ChatGPT said:

mjlab 是一个专门用于机器人强化学习（**Robot Reinforcement Learning**）的开源训练框架。

它的核心价值是：让你在 **GPU** 上同时模拟大量机器人，通过强化学习训练出运动控制 **Policy**，然后把训练好的 **Policy** 部署到真实机器人。

它不是语言模型，也不是机器人操作系统，而是连接物理仿真、强化学习和机器人控制策略的工程框架。

1. mjlab、MuJoCo、PPO 到底是什么关系？

这三个概念很容易混淆，但它们负责不同的事情。

技术	定位	作用
MuJoCo	Physics Engine	计算机器人的重力、碰撞、摩擦、关节运动等物理行为
mjlab	Robot Learning Framework	组织训练环境、观察、动作、奖励、并行仿真
PPO	RL Algorithm	通过奖励优化神经网络 Policy
ONNX	Model Format	导出训练好的 Policy，供机器人运行

mjlab 的设计参考了 NVIDIA Isaac Lab，并使用 MuJoCo Warp 加速 GPU 物理仿真。

GitHub +1

2. 整个训练架构

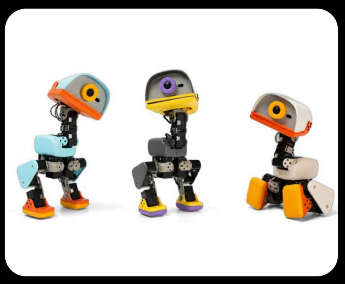


简单理解：

- **MuJoCo** 负责让虚拟机器人符合物理规律。
- **mjlab** 负责组织机器人如何学习。
- **PPO** 负责真正更新 Policy。
- **ONNX** 负责把成果带到实际机器人。

3. 你给的网站还有一个重要细节

你现在看的其实是 **XGO-Duck**，和前面讨论的 Pollen Robotics Microduck 不是同一个产品。


根据 XGO-Duck 官方网页：

- 训练端采用 **mjlab + PPO**，需要支持 CUDA 的 GPU 电脑。
- 机载计算平台是 **Arduino Uno Q**，不是 Radxa Zero 3W。
- Linux 侧以 50 Hz 执行 ONNX Policy。
- STM32 侧以 100 Hz 执行底层控制。
- 提供行走、起身和叼取等训练与运行能力。

代码与结构 STL...

所以，虽然两种机器人采用相似的强化学习思路，但在硬件架构、软件实现和开发生态方面有明显区别。

4. 对你而言，mjlab 值得学吗？

非常值得，特别是如果你希望通过这个项目转向 **Robotics Software** 或 **Embodied AI**。

而且 mjlab 不只适用于鸭子机器人。它可以训练双足行走、机器人手臂操作、动作模仿等任务，官方示例甚至包括 Unitree G1 人形机器人。

GitHub

我建议你按这样的顺序学习：

1. 先用 mjlab 运行官方 Demo，理解机器人环境。
2. 学习 Observation、Action、Reward 和 Termination 的具体实现。
3. 修改 Reward Function，重新训练 PPO。
4. 比较新旧 Policy 的稳定性。
5. 导出 ONNX，在实际机器人上部署和验证。

如果你是打算为正式机器人岗位积累经验，第三、第四、第五步的价值最高。

你可以从这些资源入手：

- mjlab 官方 GitHub ↗
- mjlab 官方文档 ↗
- mjlab 研究论文（2026） ↗
- XGO-Duck 强化学习仓库 ↗

还有一个很值得深入的问题： mjlab 训练的时候，假设同时模拟 4,096 只鸭子，PPO 是训练出 4,096 个 Policy，还是所有鸭子共享一个 Policy？

答案通常是共享一个 **Policy**。4,096 个并行环境负责提供大量不同的训练经验，共同优化同一组网络参数。

理解这一点，基本就开始真正理解现代机器人强化学习为什么需要 GPU，以及 mjlab 为什么有价值了。

You said:

在这整件事里，我的 jetson orio nano 能办演什么最佳角色

Worked for 38s >

ChatGPT said:

You said:

还是没懂 jetson 可以在 microduck这个生态里 具体处于什么位置 远端智能决策？
那现有的 duck 的 摄像头 在microduck里 难道是 policy解决的 视频识别么？ 因为我看在 演示里 这个机器人是有视频识别 和音频识别能力的

Worked for 37s >

ChatGPT said:

对，你现在理解的方向是正确的：**Jetson Orin Nano** 可以作为 **Microduck** 的远端智能决策系统（**Remote AI Brain**）。

但关键是：**Microduck** 的摄像头识别、声音识别和强化学习 **Policy**，并不是同一件事情，也不一定由同一个模型执行。

我刚刚重新查看了 Pollen Robotics 的 Microduck 官方架构文档，发现它其实已经考虑过外部 AI 的接入，而且与你的 Jetson 非常契合。

1. 先回答最关键的问题：视频识别是不是 Policy 做的？

通常不是。**Microduck** 的运动 **Policy** 主要处理运动控制，而不是直接识别视频中的物体。

例如，假设你对 Microduck 说：

“找到桌上的红色小球，走过去。”

这其实涉及三个完全不同的智能能力。

智能类型	需要解决的问题	对应技术
Perception（感知）	摄像头里有没有红球？它在哪里？	CV、Object Detection
Decision（决策）	要不要靠近？向什么方向走？	行为规划、规则或 AI Agent
Motion Policy（运动策略）	为了向左转，15 个舵机应该怎么运动？	强化学习训练的 Policy

运动 **Policy** 不必理解什么叫红色小球。

它只需要接收到类似下面的命令：

```
forward_velocity = 0.1 m/s
```

```
turn_rate = 0.2 rad/s
```

然后结合 IMU 和关节反馈，计算下一次控制循环的电机动作。

所以 Microduck 展示出的视觉能力和运动能力是可以分离的。

2. Microduck 的官方架构到底是什么？

这是官方代码库中描述的核心服务：



官方明确把摄像头和媒体处理放在 `mediad`，把强化学习 Policy 和电机控制放在 `robotd`。

原因是即使摄像头或识别服务崩溃，也不应该影响机器人的平衡和运动安全。

GitHub +1

也就是说，你看到机器人能走路、能看东西、能发声音，不意味着这些能力来自一个统一的 AI 模型。

3. Jetson 具体应该插在哪里？

答案是：接在 **Microduck** 的 `mediad` 和控制 **API** 外面，充当外部视觉、语音理解与自主决策服务。



Jetson 是建议增加的独立服务，并非 Microduck 出厂自带的组件。

实际上，官方 FAQ 已经明确提出：

当模型太大、无法在 Radxa Zero 3W 上运行时，可以把模型放在外部计算机，使用 WebRTC 传输摄像头画面，再由外部程序通过机器人 API 控制动作。

官方推荐的外部算力示例是 Hugging Face Space。你的 Jetson 完全可以成为另一个外部运行目标。

GitHub

因此，你甚至不需要在鸭子上再安装摄像头。直接使用鸭子自身的摄像头，把视频流传给 **Jetson** 即可。

4. 你看到的视频识别、音频识别演示，到底来自哪里？

这里有一个重要发现。

目前的 Microduck 生态中，至少要区分官方 Microduck 与第三方扩展项目。

能力	**Pollen 官方 Microduck**	**第三方 DuckBuddy**
强化学习行走	已实现	已实现并针对不同硬件重新训练
摄像头视频传输	已实现	已实现
本地视觉识别	存在宠物触摸检测等特定能力，通用感知仍在发展	人脸、运动检测、二维码识别
语音识别	不能确认已有完整的通用 ASR 功能	使用 SenseVoice
发声与合成声音	已实现	已实现
多模态 AI 理解	有远端扩展设计	可以调用远端多模态模型

Pollen 官方项目在 2026 年 8 月的 Roadmap 中明确表示，自主行为大脑（Autonomous Brain）仍未完全移植；摄像头视频传输已经在实体硬件上运行。

GitHub

而我发现另一个值得你看的项目：

DuckBuddy — Microduck 第三方扩展项目 ↗

它基于 Microduck 开源设计，增加了：

- `voiced`：离线语音识别、声音方向检测。
- `visiond`：人脸与运动检测。
- `lifed`：自主行为、状态机、任务管理。

- `duckbrain`：可选的远端多模态 AI 大脑。

其中 `duckbrain` 几乎就是我建议 **Jetson** 承担的那个角色。只需要将这种外部 AI 服务部署到你的 Jetson 上，并适配相应的视频、音频与控制接口。需要注意，DuckBuddy 使用不同的硬件和软件扩展，并不能直接假定与官方 Microduck 完全兼容。

GitHub

5. 所以 Jetson 的真正价值是什么？

如果只是让 Microduck 行走、起身、转弯，**Jetson** 没有必要参与。

如果只是识别简单的人脸或声音，Radxa 也可能通过轻量模型完成。

但如果你希望：

“Duck，看看桌子上有什么。”

机器人回答：

“我看到了一个红色杯子和一本书。”

接着你说：

“走到红色杯子附近，但不要碰到它。”

此时 Jetson 可以执行多模态理解、目标检测与行为规划，然后持续根据新的视频反馈调整机器人的高级运动命令。

而 Microduck 本地的 RL Policy 仍然只负责如何安全地迈步。

所以 **Jetson** 不是替代 **Policy**，而是在 **Policy** 之上增加一层真正的智能感知和行为决策。

这也是我建议你做的项目方向：**Microduck + Jetson Orin Nano**，构建一个具有自主视觉感知和任务决策能力的 **Embodied AI** 系统。

最后，我想确认一个会直接影响我们下一步技术分析的细节：你说的能进行视频识别、音频识别的演示，是 **Pollen Robotics** 官方 **Microduck** 的演示，还是 **XGO-Duck / DuckBuddy** 的演示？它们外观与技术路线有相似之处，但 AI 功能的实现方式并不相同。

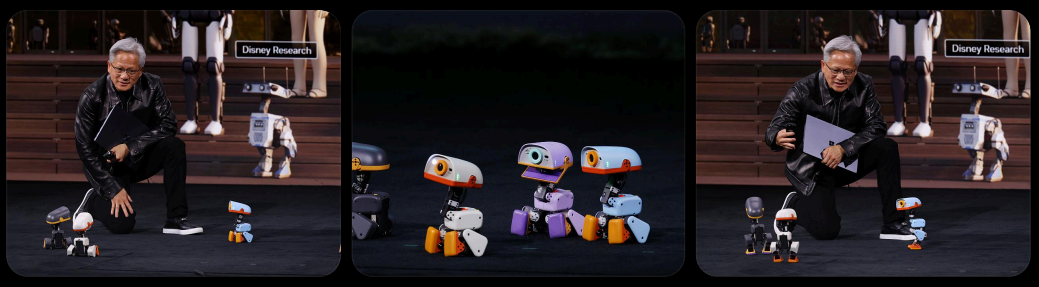
You said:

我指的是 Pollen Robotics 官方 Microduck 的演示 以及黄仁勋在 几天前 GTC Berlin大会上与四个microduck的互动 看到的

Worked for 39s >

ChatGPT said:

明白了。你说的是 **2026** 年 **10** 月 **21** 日 **NVIDIA GTC Berlin** 大会上，黄仁勋与四只 **Pollen Robotics Microduck** 互动的演示。



我找到 NVIDIA 官方对这次演示的报道，也核对了 Microduck 的代码架构。

这里有个很重要的发现：

这次演示实际上展示了两个不同层次的 **AI**：一个是训练机器人运动技能的 **Physical AI** 系统；另一个是让机器人接收指令并选择行为的控制系统。但演示本身不能证明四只鸭子各自在本地运行了语音识别或视觉大模型。

1. 黄仁勋的演示究竟展示了什么？

NVIDIA 官方明确介绍了这条训练流程：



这条流程是 NVIDIA 对现场演示的官方解释。

NVIDIA Blog +1

黄仁勋确实对机器人发出了坐下、站起、睡觉、翻滚等口头指令。

但注意，这证明了机器人能够执行相应动作，并不能单凭视频判断指令是由机器人自己的麦克风识别、场外控制程序识别，还是演示人员通过其他方式触发的。现场几只机器人的反应也不是完全一致。

StockAnalys… +1

所以我们不能直接得出“Microduck 内置了完整语音大模型”的结论。

2. Microduck 自己究竟能看懂、听懂多少？

官方 Microduck 的硬件确实包括前置摄像头、麦克风、扬声器、ToF 深度传感器以及 RK3566 AI 加速器。

Pollen Robotics

但硬件支持与软件已经实现的能力，要分开理解：

功能	官方实现情况	使用 **RL Policy** 吗？
行走、坐下、翻滚、起身	已提供预训练动作	是
摄像头画面传输	已实现 WebRTC 视频流	否
识别其他鸭子	已有检测器相关代码与设计	否，属于视觉检测
检测被抚摸的声音	已有 ONNX 分类模型	否，属于音频分类
识别任意自然语言指令	尚不能确认官方完整支持	否，需要 ASR 等组件
理解复杂场景并自主规划	不属于基础运动 Policy	通常需要额外感知和规划模块

官方代码中甚至提供了一个声音分类模型 `pet_detect.onnx`，用于判断机器人是否被触摸；它不是语音转文字模型。

GitHub +2

这意味着：**Microduck** 并非完全没有本地感知智能，只是它的本地视觉、音频模型与运动 **Policy** 是相互独立的。

3. Jetson 究竟应该放在哪里？

我现在可以给你一个更加准确的架构定位：



这是我建议你开发的集成架构，不是 GTC Berlin 舞台系统的已确认拓扑。

最关键的是，**Microduck** 官方软件架构已经为这种外部 **AI** 方式提供了基础。

官方设计文件明确支持通过 WebRTC 将摄像头、麦克风音频传递给远端客户端，并利用 DataChannel 发送控制指令。同时，`robotd` 保留对动作安全和关节控制的最终决定权。

GitHub

所以你可以开发这样一条完整的链路：

1. 你说：“Duck，走到红色小球旁边。”
2. Microduck 把麦克风音频、摄像头视频发送给 Jetson。
3. Jetson 的 ASR 把语音转为文本，视觉模型识别小球。
4. Jetson 的 Planner 根据小球的位置计算运动目标。
5. Jetson 通过 API 向 Microduck 发送前进和转向指令。
6. Microduck 的本地 RL Policy 控制舵机，实现行走与平衡。
7. Jetson 不断读取新画面，判断是否到达目标。

这里 Jetson 并不执行运动 Policy，也不需要直接操作电机。

4. 那么 **GTC Berlin** 与你的 **Jetson** 最有价值的联系是什么？

GTC 演示中，NVIDIA 把 RTX Spark 用作 **AI Agent** 与机器人开发、训练工作流的协调平台，大量训练工作仍交给云端 GPU。

你的 Jetson 不具备 RTX Spark 同等级的计算资源，但可以承担类似架构中的一部分职责：

- 运行机器人开发与测试工具。
- 为机器人提供视觉、语音及任务规划服务。
- 接收机器人 Telemetry，记录实验数据。
- 管理已经训练完成的 Policy，执行部署前验证。
- 与云 GPU 的训练工作流集成。

这意味着你可以让 Jetson 成为一个 **Robot Intelligence Server + Robotics Development Node**。

但我建议不要一开始就试图复刻 GTC 整套 NVIDIA 开发演示。先让 Jetson 利用 Microduck 自带的摄像头识别一个目标，再通过控制 API 让鸭子走向目标，就已经能实现一个真正的视觉闭环自主机器人系统。

最后澄清一个容易产生误解的地方： GTC Berlin 的舞台演示，证明 Microduck 能执行训练过的动作并与人的指令形成互动；但我没有找到 NVIDIA 或 Pollen 的公开资料，足以确认当晚具体使用了哪种 ASR、是否在每只鸭子本地运行、有没有舞台控制系统协助。

因此，不能把舞台上的语音互动直接视为其出厂即具备完整多模态理解能力的证据。

你可以直接看 NVIDIA GTC Berlin 官方演示报道 ↗ 和 Microduck 官方架构设计 ↗。

对于你未来的机器人开发经历，我反而觉得最有价值的方向已经清楚了：用 **Jetson** 给 **Microduck** 建立一个可复现、可测试的多模态自主决策层，同时利用 **mjlab** 研究与改进它原有的运动 **Policy**。 这两项工作既相互独立，又能整合成完整的 Physical AI 工程项目。

You said:

我很欣赏你的规划
但是我还是想先确认一下 microduck Radxa zero w3 以及他本地的那套东西 到底有没有承担音视频多模态推理的能力，还是说 本身就需要远端AI做推理，前端知识个15舵机的姿态控制

Worked for 16s >

ChatGPT said:

现在可以给你一个比较明确的答案了。我重新查了 Pollen Robotics 官方的运行代码、配置文件和架构文档。

结论：**Microduck** 的 **Radxa Zero 3W** 确实承担了本地 **AI** 推理，而且不仅仅是 **15** 个舵机的运动控制。但它运行的是多个专用的小模型，而不是一个能够通用理解音视频和自然语言的大型多模态模型。

因此，它既不是完全依赖远端 AI 的机器人，也不是完全自主的通用多模态机器人。

1. 官方代码证明：**Microduck** 已具备这些本地推理能力

能力	实际执行位置	官方实现情况
站立、行走、转身、起身	Radxa CPU	本地 RL Policy
识别其他 Microduck	Radxa NPU 或 CPU	`duck_detect.rknn` / `duck_detect.onnx`
检测被抚摸的声音	Radxa CPU	`pet_detect.onnx`
摄像头视频采集与编码	Radxa 硬件视频编码器	WebRTC / H.264
麦克风音频采集	Radxa 音频系统	本地支持
判断前方障碍距离	ToF 传感器 + 本地处理	深度数据
理解任意自然语言对话	尚未确认有完整本地实现	需要额外模型或外部服务
通用视觉问答、跨模态推理	尚未确认有完整本地实现	更适合外部 AI

其中有两个非常具体的证据。

第一个：**Microduck** 官方有真正的本地图像识别模型。

配置中明确写着：

- `duck_detect.rknn`：通过 Rockchip NPU 运行。
- `duck_detect.onnx`：通过 CPU 运行。
- 默认优先 NPU，不可用时尝试 CPU。
- 模型独立发布，可以单独升级。

也就是说，Microduck 可以自己看摄像头画面，识别画面中是否出现另一只鸭子，而不是必须把图像传给云端。这个检测器默认关闭，官方设置的检测频率是 2 Hz，以控制发热。

GitHub +1

第二个：**Microduck** 本地确实有音频分类模型。

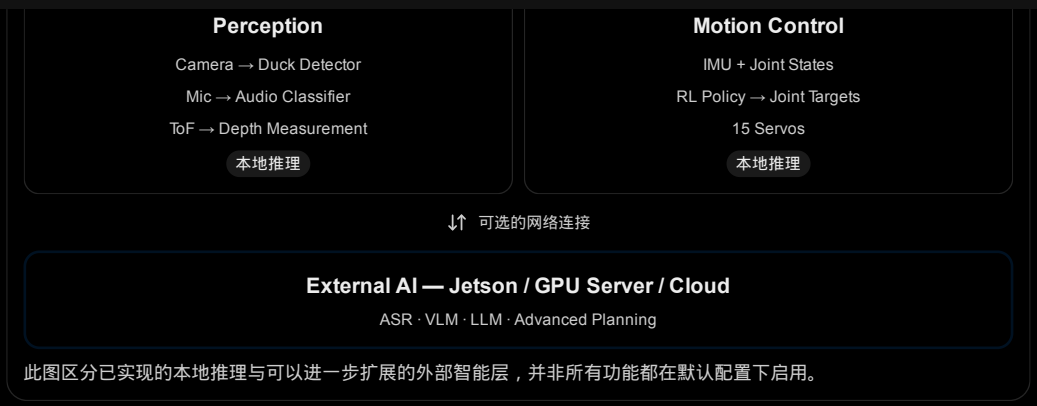
官方提供 `pet_detect.onnx`，利用麦克风输入判断是否存在类似抚摸的声音事件，并触发相应反应。

但它不等同于 Speech Recognition（语音识别）：识别一种声音事件和理解“请向左走”是两种不同复杂度的任务。这个功能也属于可选配置。

GitHub

2. 实际架构应该这样理解





此图区分已实现的本地推理与可以进一步扩展的外部智能层，并非所有功能都在默认配置下启用。

3. 那么它的本地 AI 能力到底有多强？

Radxa Zero 3W 使用 RK3566 SoC，其中的 NPU 约为 **0.8 TOPS INT8**。

官方 FAQ 对它的能力边界说得很直接：这颗 NPU 适合运行一个小型检测模型，更复杂的模型建议放到外部设备。

GitHub

所以要区分两个概念：

专用感知模型（**Specialised Perception Model**）

输入一张图片，输出“检测到一只鸭子，位置为 x、y”。

这种功能完全可以由 Radxa 本地承担。

通用多模态推理（**General Multimodal Reasoning**）

输入视频和语音，理解：

“那只红色鸭子是不是正在靠近我？如果是，就向后退一点，并告诉我发生了什么。”

这需要目标识别、语音理解、时间信息整合、行为规划，可能还需要 LLM/VLM。现有官方资料不足以证明出厂版本已经在本地完整实现这一套。

4. 这反而改变了 Jetson 的最佳定位

之前我把 Jetson 描述成 Microduck 的远端智能大脑。现在更精确的说法应该是：

Jetson 是 **Microduck** 现有本地智能的可选增强层，而不是它正常运行的必要组成部分。

Microduck 本身已经具备运动智能、有限的视觉感知及声音事件识别。

Jetson 可以进一步提供通用语音理解、视觉场景分析、目标跟踪、多步骤任务规划等能力，并将高级意图交给 Microduck 执行。

但是，对于只需要“看见另一只鸭子并转头”的行为，可能完全没有必要经过 Jetson。本地检测器配合运动 API 就足够了。

5. 官方资料中有一段尤其值得你看

Pollen Robotics 在架构设计中明确提出：

`mediad` 应该在靠近摄像头的位置执行感知推理，然后把结果（例如物体坐标、是否检测到人）交给 `robotd`，而不是把完整图像送进 50 Hz 运动控制循环。

这符合典型的边缘机器人架构原则：感知服务处理原始数据，运动控制服务只接收必要的状态特征与行为意图。

GitHub

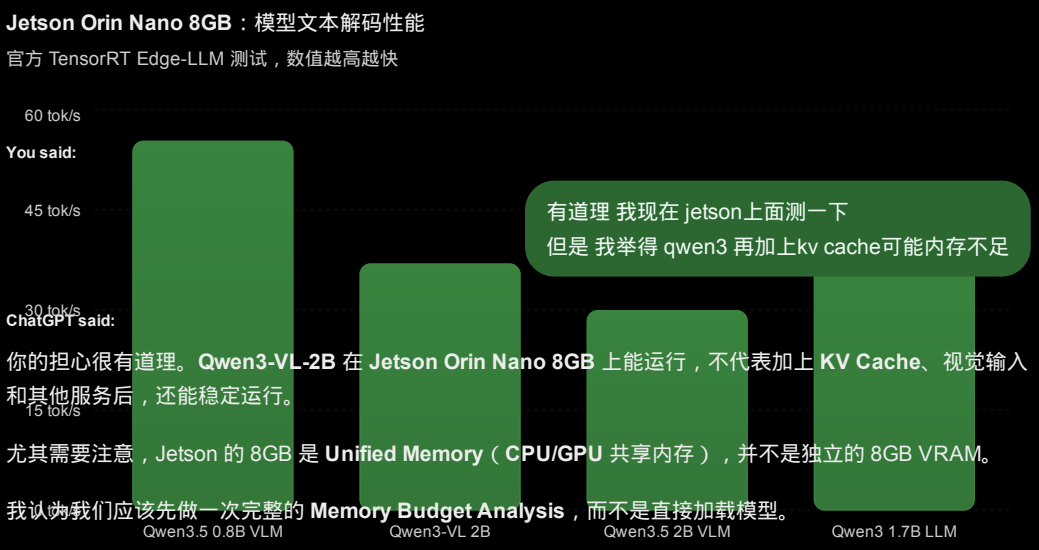
需要注意，这份架构文档包含目标设计，不能把其中每个细节都视为已经交付的软件功能。不过，鸭子检测器和声音分类器的配置已经能在代码中找到。

你可以直接查看：

- 官方配置：视觉检测器与音频分类器 ↗
- 官方 FAQ：什么时候需要外部 AI ↗
- 官方架构：本地感知与运动控制边界 ↗

我的最终判断：**Microduck** 本身是一台具有真实本地 **AI** 推理能力的机器人，而不是只有 **15** 个舵机、所有智能都在云端的遥控设备。但它的本地 **AI** 主要属于专用、轻量模型，而非通用多模态大模型。

如果未来你想让它真正做到“看懂环境、听懂自然语言、自主决定下一步做什么”，Jetson Orin Nano 才开始展现它不可替代于 Radxa 的计算价值——尽管 Jetson 自身的 8GB 内存也会限制可运行的模型规模。



You said:

有道理 我现在 jetson上面测一下
但是 我举得 qwen3 再加上kv cache可能内存不足

ChatGPT said:

你的担心很有道理。**Qwen3-VL-2B** 在 **Jetson Orin Nano 8GB** 上能运行，不代表加上 **KV Cache**、视觉输入和其他服务后，还能稳定运行。

尤其需要注意，Jetson 的 8GB 是 **Unified Memory**（**CPU/GPU** 共享内存），并不是独立的 8GB VRAM。

我认为我们应该先做一次完整的 **Memory Budget Analysis**，而不是直接加载模型。

1. 内存究竟花在哪里？

运行 VLM 时，内存主要被以下部分占用：

$$M_{\text{total}} \approx M_{\text{OS}} + M_{\text{weights}} + M_{\text{KV}} + M_{\text{vision}} + M_{\text{runtime}} + M_{\text{other}}$$

其中：

组成	对内存的影响
Linux OS + 后台服务	持续占用
Model Weights	INT4 量化后显著降低
KV Cache	随上下文 Token 数增长
Vision Encoder / 图像中间张量	受图像分辨率和视觉 Token 数影响
Runtime / CUDA Workspace	与推理引擎有关
Whisper、YOLO 等服务	如果同时运行，还需额外内存

最容易被低估的是 **KV Cache** 和视觉 **Token** 数量。

KV Cache 的近似公式是：

$$M_{\text{KV}} = 2 \times L \times H_{kv} \times D_h \times T \times B$$

这里：

- L：Transformer 层数。
- H_{kv}：KV Head 数量。
- D_h：每个 Head 的维度。
- T：缓存的 Token 数量，包括图像对应的 Token。
- B：每个元素的字节数，例如 FP16 为 2 bytes。

这也说明不能简单地根据模型是 2B，就断定 KV Cache 一定很大或很小。GQA、缓存精度和上下文长度都会影响结果。

2. 我们可以算出 **Qwen3-VL-2B** 的 **KV Cache** 大小

我核对了它在 Hugging Face 上的实际模型配置：

Qwen/Qwen3-V…

- Transformer Layers：28
- KV Heads：8
- Head Dimension：128
- KV Precision：假设 FP16（2 bytes）
- Batch Size：1

代入公式：

$$M_{\text{KV/token}} = 2 \times 28 \times 8 \times 128 \times 2 = 114688 \text{ bytes}$$

即每个 Token 约占 **112 KiB**。

Qwen3-VL-2B KV Cache 估算器

Context Length 2,048 tokens

理论 FP16 KV Cache

224 MiB

Batch = 1，未包含缓存分配开销及其他模型内存

这说明一件重要的事：

在 **2K–4K Context** 下，**KV Cache** 本身未必是最严重的问题。模型权重、视觉编码器、**Runtime Workspace** 以及操作系统共同占用的内存，更可能成为主要约束。

NVIDIA 在 Jetson Orin Nano 8GB 上测得 Qwen3-VL-2B INT4 推理约占 4.4GB GPU Memory，不过这个数字与测试引擎的配置和内存分配策略有关，不能直接用它加上上面的 KV Cache 估算，否则可能重复计算。

TensorRT Edg…

3. 对 **Microduck**，我建议先限制 **Context**，而不是放弃 **2B** 模型

因为我们要的是机器人执行短任务，不是维持几万 Token 的长对话。

我的第一版配置建议：

参数	起始配置
Model	Qwen3-VL-2B-Instruct
Quantisation	INT4 AWQ
Context	2048 tokens
KV Cache	FP16
Batch Size	1
Input	单张图像，按需分析
Output	最多 64–128 tokens
Chat History	尽量不保留，必要时摘要
其他 AI 服务	初期暂不同时加载

然后逐步测试 4096 Context、连续图像输入、Whisper 并发运行等场景。

如果 2B 实在不稳定，我更推荐退到 **Qwen3.5-0.8B VLM**，而不是通过大量 Swap 强行运行 2B。

NVIDIA 的基准显示 Qwen3.5-0.8B VLM 在相同设备上约使用 2.6–2.8GB GPU Memory，能给其他机器人服务留下更多空间。

TensorRT E… +1

4. 在 Jetson 上先测一下实际可用内存

先运行：

```bash
free -h
sudo tegrastats --interval 1000
```

另一个终端运行：

```bash
cat /etc/nv_tegra_release
cat /etc/os-release
df -h /
```

我希望先确认三件事：

1. 你的 Jetson 当前还剩多少物理 RAM。
2. JetPack / L4T 是否适合 NVIDIA 的 TensorRT Edge-LLM。
3. 恢复后的系统有没有大量后台服务占用内存。

尤其考虑到你这台 Jetson 前段时间曾出现黑屏启动问题，我们应先确认系统状态稳定，再部署模型。

我的建议是：先挑战 **Qwen3-VL-2B + 2048 Context**。如果内存紧张，再试 **Qwen3.5-0.8B**，而不是一开始就牺牲太多视觉理解能力。

这样后面我们还能进一步评估一个更有意思的问题：让 Microduck 在本地执行轻量目标检测，只在需要复杂语义理解时才把关键帧传给 Jetson，从系统层面减少 VLM 的推理频率和内存压力。

You said:

TensorRT Edge-LLM 这是什么 没听过

Worked for 8s

ChatGPT said:

TensorRT Edge-LLM 是 **NVIDIA** 专门为 **Jetson** 等边缘计算设备设计的 **LLM / VLM** 高性能推理框架。

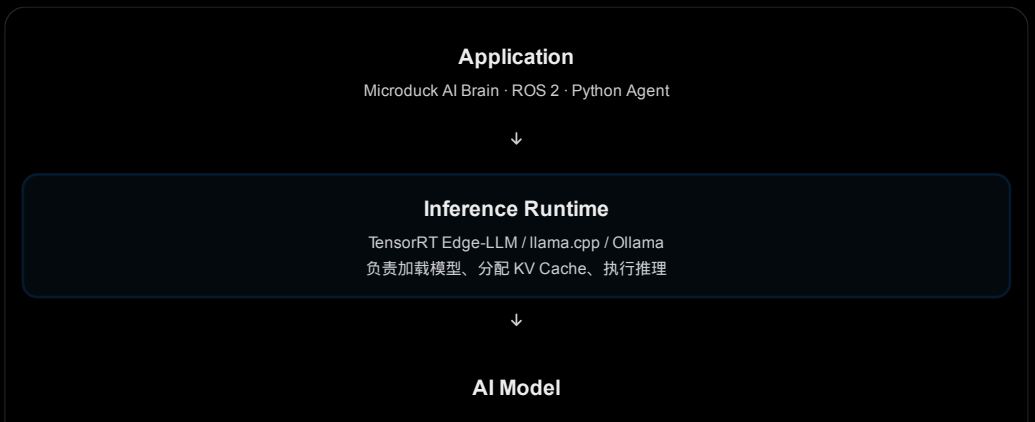
它比较新，而且与你现在准备做的 **Jetson Orin Nano + Qwen3-VL + Microduck** 项目非常相关。

你可以这样理解：

> **Qwen3-VL** 是 **AI** 模型，**TensorRT Edge-LLM** 是让这个模型在 **Jetson** 的 **NVIDIA GPU** 上高效运行的推理引擎。

它和 llama.cpp、Ollama 属于同一技术层次，但优化目标不同。

1. TensorRT Edge-LLM 在 AI 技术栈中的位置



Qwen3-VL-2B · 量化权重 + Vision Encoder

↓

NVIDIA Acceleration

TensorRT · CUDA · GPU

↓

Jetson Orin Nano 8GB

更准确地说，推理引擎会将模型计算转换为 GPU 可执行的运算；模型权重并不是在运行时简单地“传过”推理框架。

2. 为什么 NVIDIA 要专门开发 Edge-LLM？

NVIDIA 原来已经有 TensorRT 和 TensorRT-LLM。

但面向数据中心的高吞吐量 LLM 服务与面向机器人的推理有不同要求：

维度	**TensorRT-LLM**	**TensorRT Edge-LLM**
主要定位	GPU 服务器、大规模 LLM 服务	Jetson、DRIVE 等边缘计算
核心目标	高吞吐、批处理、高并发	低延迟、紧凑运行时、嵌入式部署
Runtime	功能丰富的推理服务栈	轻量 C++ Runtime
多模态	支持	支持 LLM、VLM、音频及动作模型
你的 Jetson	需考虑特定版本支持	更值得优先研究

NVIDIA 已将 Jetson Orin 列为 Edge-LLM 的支持目标。它的运行时主要采用 C++，可以减少生产部署对 Python 的依赖。

GitHub +1

3. 它对 8GB 内存有什么帮助？

这正好对应你刚刚提出的 KV Cache 问题。

TensorRT Edge-LLM 主要通过几种机制优化资源使用：

- **INT4 Weight Quantization**：大幅降低模型权重占用。
- **Paged Attention**：更灵活地管理 KV Cache，减少某些场景的内存浪费。
- **KV Cache Reuse**：重复利用符合条件的前缀缓存。
- **TensorRT Engine Optimization**：优化计算图、内核和推理执行过程。

但特别注意：它并不能让 **KV Cache** 消失，也不能保证所有模型都比 **llama.cpp** 更省内存。

还有一个容易踩的坑：虽然 Edge-LLM 的通用文档提到 FP8 KV Cache，但你的 Jetson Orin 属于 Ampere 架构，官方支持矩阵强调 Orin 的 FP16、INT8、INT4 执行精度，不应直接假设所有 FP8 优化都可用。

TensorRT E… +1

4. TensorRT Edge-LLM 与 Ollama、llama.cpp 怎么选？

特点	**Ollama**	**llama.cpp**	**TensorRT Edge-LLM**
上手难度	低	中	高
模型格式	通常 GGUF	GGUF	TensorRT Engine
GPU 加速	支持	支持	NVIDIA 专门优化
自定义内存配置	中	高	高
模型兼容广度	很广	很广	取决于官方支持
面向正式嵌入式部署	可以	可以	核心设计目标
适合第一天验证	很适合	很适合	需要准备环境

对你的情况，我会建议分两步：

先用 llama.cpp 量化模型，验证 Qwen3-VL-2B 是否能在 Jetson 8GB 上稳定完成图像理解；然后使用 TensorRT Edge-LLM 部署相同或等效精度的模型，比较内存、首 Token 延迟、生成速度和视觉处理延迟。

这样我们可以真正判断 NVIDIA 的专用推理引擎对你的机器人场景有多大收益，而不是只比较营销基准。

5. TensorRT Edge-LLM 是怎样部署模型的？

它不一定像 Ollama 那样直接下载模型就运行。

标准流程更接近一个编译和部署管线：



最新版还提供实验性的直接从 Checkpoint 构建 Engine 的路径，但成熟度与支持范围要以具体版本为准。

GitHub

6. 对你的 Microduck 项目，有什么实际价值？

举个非常具体的例子：

Microduck 摄像头拍摄一张图片，通过 Wi-Fi 发送给 Jetson。

Jetson 上的 TensorRT Edge-LLM 运行 Qwen3-VL，返回：

JSON

```
{
  "objects": ["cup", "book"],
  "target": "cup",
  "relative_position": "left",
  "recommended_action": "turn_left"
}
```

然后你设计的 Robot Planner 对结果进行验证，生成机器人的高级运动指令。

这样 **Edge-LLM** 就相当于 **Microduck** 外部多模态智能服务的推理引擎，而不是运动控制器。

7. 下一步怎么实践？

我建议先验证软件兼容性，因为你这台 Jetson 之前曾使用 L4T R36.4.7。

在 Jetson 上执行：

Bash

```
cat /etc/nv_tegra_release
cat /etc/os-release
free -h
```

之后按照 NVIDIA 的支持矩阵选择匹配的 TensorRT Edge-LLM 版本，不要直接安装最新包而忽略 JetPack/CUDA/TensorRT 依赖。

官方资料：

- TensorRT Edge-LLM GitHub ↗
- 官方文档与 Quick Start ↗
- NVIDIA Jetson 部署教程 ↗

我比较看好这个技术栈的一个原因是：它非常贴近工业机器人领域真正需要的模型部署、低延迟推理、资源约束与硬件优化问题。

对你的职业发展而言，不只是学会运行 Qwen，而是能够展示你如何在只有 8GB 共享内存的边缘设备上，将多模态模型优化并集成到真实机器人控制系统中。

You said:

那我现在是不是可以 在jetson上先装上tensorRT edge-llm？

Worked for 13s

ChatGPT said:

可以，但我建议先检查 **Jetson** 的系统版本，再决定安装哪个 **TensorRT Edge-LLM** 版本。不要直接安装最新版。

我刚核对了 NVIDIA 官方文档，发现一个对你的设备非常重要的兼容性问题：

TensorRT Edge-LLM	**Jetson Orin Nano** 系统要求
v0.9.0	支持 JetPack 6.2+，CUDA 12.6
v0.11.0	官方支持矩阵列出 JetPack 7.2，CUDA 13.2

资料：

TensorRT E... +1

。

你之前的 Jetson 使用的是 **L4T R36.4.7**，属于 JetPack 6 系列环境。因此，如果现在仍然是这个系统，v0.9.0 是更合理的初步候选，而不是直接尝试 v0.11.0。

第一步：检查你的 **Jetson**

先执行以下命令。它们都是只读检查，不会修改系统。

Bash

```
# Jetson Linux / L4T
cat /etc/nv_tegra_release

# Ubuntu
cat /etc/os-release

# CUDA
nvcc --version

# TensorRT
dpkg -l | grep -E 'tensorrt|libnvinfer'

# RAM
free -h
```

```
# Disk
df -h /

# Architecture
uname -m
```

如果你希望一次复制运行，也可以直接执行：

Bash

```
echo "===== Jetson OS ====="
cat /etc/nv_tegra_release
cat /etc/os-release | head -n 5

echo "===== CUDA ====="
nvcc --version

echo "===== TensorRT ====="
dpkg -l | grep -E 'tensorrt|libnvinfer'

echo "===== Memory ====="
free -h

echo "===== Storage ====="
df -h /
```

第二步：根据检查结果选择安装路线

- ✅ 如果仍是 **JetPack 6.2 / CUDA 12.6**
  优先研究 TensorRT Edge-LLM v0.9.0 的源码编译路线。确认精确依赖版本后再安装。
- ✅ 如果已升级至 **JetPack 7.2 / CUDA 13.2**
  可以考虑官方最新 v0.11.0 安装流程，包括预编译 Python Wheel 或 C++ Runtime。
- ⚠️ 如果 **CUDA** 或 **TensorRT** 缺失
  先修复 JetPack 的运行环境。不要混装其他 JetPack 版本的 CUDA/TensorRT 软件包。

第三步：准备安装目标

我建议第一轮只做三个目标：

1. **Runtime Installation**：成功安装并加载 TensorRT Edge-LLM。
2. **Basic Inference**：用一个小型模型完成文本推理。
3. **VLM Inference**：加载量化视觉模型，测试摄像头图片理解。

正式运行 Qwen3-VL-2B 前，还要检查空间。NVIDIA 的源码构建工作流建议预留约 20–50GB 磁盘用于 ONNX 和 TensorRT Engine 文件。

TensorRT Edg…

考虑到你之前 Jetson 的黑屏故障和系统修复经历，我建议暂时不升级 **JetPack**、不覆盖现有 **CUDA/TensorRT**，也不运行任何系统级安装脚本。

相关官方文档：

- TensorRT Edge-LLM v0.9.0 安装指南 ↗
- 最新兼容性矩阵 ↗

现在先把上面第二段代码在 **Jetson** 上运行，把输出贴给我。 我们先锁定版本，再给你的 Orin Nano 制定对应的安装命令，最终在 8GB 共享内存条件下测试 Qwen VLM 的真实表现。

You said:

```
# R36 (release), REVISION: 4.7, GCID: 42132812, BOARD: generic,
EABI: aarch64, DATE: Thu Sep 18 22:54:44 UTC 2025
# KERNEL_VARIANT: oot
TARGET_USERSPACE_LIB_DIR=nvidia
```

```
TARGET_USERSPACE_LIB_DIR_PATH=usr/lib/aarch64-linux-gnu/nvidia
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
-bash: nvcc: command not found
ii  libnvinfer-bin                                  10.3.0.30-1+cuda12.5
arm64        TensorRT binaries
ii  libnvinfer-dev                                  10.3.0.30-1+cuda12.5
arm64        TensorRT development libraries
ii  libnvinfer-dispatch-dev                         10.3.0.30-1+cuda12.5
arm64        TensorRT development dispatch runtime libraries
ii  libnvinfer-dispatch10                           10.3.0.30-1+cuda12.5
arm64        TensorRT dispatch runtime library
ii  libnvinfer-headers-dev                          10.3.0.30-1+cuda12.5
arm64        TensorRT development headers
ii  libnvinfer-headers-plugin-dev                   10.3.0.30-1+cuda12.5
arm64        TensorRT plugin headers
ii  libnvinfer-lean-dev                             10.3.0.30-1+cuda12.5
arm64        TensorRT lean runtime libraries
ii  libnvinfer-lean10                               10.3.0.30-1+cuda12.5
arm64        TensorRT lean runtime library
ii  libnvinfer-plugin-dev                           10.3.0.30-1+cuda12.5
arm64        TensorRT plugin libraries
ii  libnvinfer-plugin10                             10.3.0.30-1+cuda12.5
arm64        TensorRT plugin libraries
ii  libnvinfer-samples                              10.3.0.30-1+cuda12.5
all          TensorRT samples
ii  libnvinfer-vc-plugin-dev                        10.3.0.30-1+cuda12.5
arm64        TensorRT vc-plugin library
ii  libnvinfer-vc-plugin10                          10.3.0.30-1+cuda12.5
arm64        TensorRT vc-plugin library
ii  libnvinfer10                                    10.3.0.30-1+cuda12.5
arm64        TensorRT runtime libraries
ii  nv-tensorrt-local-tegra-repo-ubuntu2204-10.3.0-cuda-12.5 1.0-1
arm64        nv-tensorrt-local-tegra repository configuration files
ii  python3-libnvinfer                              10.3.0.30-1+cuda12.5
arm64        Python 3 bindings for TensorRT standard runtime
ii  python3-libnvinfer-dev                          10.3.0.30-1+cuda12.5
arm64        Python 3 development package for TensorRT standard runtime
ii  python3-libnvinfer-dispatch                     10.3.0.30-1+cuda12.5
arm64        Python 3 bindings for TensorRT dispatch runtime
ii  python3-libnvinfer-lean                         10.3.0.30-1+cuda12.5
arm64        Python 3 bindings for TensorRT lean runtime
ii  tensorrt                                        10.3.0.30-1+cuda12.5
arm64        Meta package for TensorRT
               total        used        free      shared  buff/cache   available
Mem:           7.4Gi       1.5Gi       384Mi        28Mi       5.6Gi       5.7Gi
Swap:          3.7Gi       2.0Mi       3.7Gi
```

Filesystem Size Used Avail Use% Mounted on
/dev/mmcblk0p1 469G 27G 423G 7% /
aarch64
…

Show more

ChatGPT said:

You said:

```
===== JetPack =====
===== CUDA packages =====
ii  cuda-compiler-12-6                      12.6.11-1
arm64       CUDA compiler
ii  cuda-cudart-12-6                        12.6.68-1
arm64       CUDA Runtime native Libraries
ii  cuda-cudart-dev-12-6                    12.6.68-1
arm64       CUDA Runtime native dev links, headers
ii  cuda-toolkit-12                         12.6.11-1
arm64       CUDA Toolkit 12 meta-package
ii  cuda-toolkit-12-6                       12.6.11-1
arm64       CUDA Toolkit 12.6 meta-package
ii  cuda-toolkit-12-6-config-common         12.6.68-1
all         Common config package for CUDA Toolkit 12.6.
ii  cuda-toolkit-12-config-common           12.6.68-1
all         Common config package for CUDA Toolkit 12.
ii  cuda-toolkit-config-common              12.6.68-1
all         Common config package for CUDA Toolkit.
===== CUDA paths =====
lrwxrwxrwx  1 root root   22 Jun 18  2025 /usr/local/cuda ->
/etc/alternatives/cuda
lrwxrwxrwx  1 root root   25 Jun 18  2025 /usr/local/cuda-12 ->
/etc/alternatives/cuda-12
drwxr-xr-x 12 root root 4096 Jun 18  2025 /usr/local/cuda-12.6
-rwxr-xr-x 1 root root 16370032 Aug 15  2024 /usr/local/cuda-
12.6/bin/nvcc
-rwxr-xr-x 1 root root 16370032 Aug 15  2024 /usr/local/cuda-
12/bin/nvcc
-rwxr-xr-x 1 root root 16370032 Aug 15  2024 /usr/local/cuda/bin/nvcc
===== GPU runtime =====
crw-rw---- 1 root video 492,   1 Oct 22 23:29 /dev/nvhost-gpu
crw-rw---- 1 root video  10, 124 Oct 12 15:44 /dev/nvmap
===== Power modes =====
[sudo] password for ksmart:
NV Power Mode: 15W
0
…
```

Show more

Worked for 18s

ChatGPT said:

很好，这次结果确认了一个关键事实：你的 **Jetson Orin Nano** 已经安装完整的 **CUDA 12.6 Toolkit**，之前不需要重装 **CUDA**。

不过，我还发现一个与 TensorRT Edge-LLM 编译直接相关的兼容性问题，需要提前处理。

1. 最新诊断结果

CUDA Toolkit 12.6 ✓ 已安装

TensorRT 10.3	已安装
GPU 设备节点	存在
Power Mode	15W / Mode 0
可用 RAM	5.7 GiB
可用存储	423GB

另外，`dpkg-query` 没有显示 `nvidia-jetpack` 元包，并不表示 CUDA、TensorRT 或 GPU 驱动不存在。你已经确认安装了主要组件，只是 JetPack 元包的状态还不明确。

2. 先解决 nvcc 路径

你已经确认：

`/usr/local/cuda-12.6/bin/nvcc`

真实存在。

因此可以先直接测试：

```bash
/usr/local/cuda-12.6/bin/nvcc --version
```

如果正常，再为当前用户设置 PATH：

```bash
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

nvcc --version
```

这只修改当前用户的 shell 环境，不会修改 CUDA 驱动或 TensorRT 软件包。

3. 发现一个 TensorRT Edge-LLM 兼容性风险

虽然 NVIDIA 官方的 v0.9 安装文档列出了 Jetson Orin + JetPack 6.2+ + CUDA 12.6 的组合，但 GitHub 已有人报告：

- TensorRT Edge-LLM **v0.9.1** 的部分代码要求 TensorRT 10.7。
- 另一些代码引用了 TensorRT 10.8 才支持的 FP4 类型。
- 你的 TensorRT 版本是 **10.3**。

因此，v0.9.1 在类似 JetPack 6.2 环境下可能编译失败。

GitHub +1

这不是你的 Jetson 配置错误，而是项目部分源码与旧版本 TensorRT 的兼容性问题。

4. 下一步建议：先准备源码，不立即编译

我们可以先把 v0.9.0 固定下来，检查对应源码与依赖。

```bash
mkdir -p ~/robotics
cd ~/robotics

git clone --branch v0.9.0 \
  https://github.com/NVIDIA/TensorRT-Edge-LLM.git

cd TensorRT-Edge-LLM

git submodule update --init --recursive
```

这里有意指定 `v0.9.0`，而不是直接拉取最新的 `main`。

接着检查工具：

```bash
cmake --version
g++ --version
git describe --tags --always
```

这几步只下载源码和检查工具，不会覆盖现有的 CUDA / TensorRT 环境。

需要说明的是：**v0.9.0** 仍然需要通过实际编译验证与 **TensorRT 10.3** 的兼容性，不能因为官方列出 JetPack 6.2 支持就认定一定成功。

5. 我们接下来真正的目标

我建议按以下顺序推进：

1. 确认 `nvcc` 可以正常运行。
2. 验证 TensorRT Edge-LLM v0.9.0 能否在当前系统完成 CMake 配置。
3. 构建 C++ Runtime，先运行一个小型文本模型。
4. 准备兼容的 Qwen VLM 量化模型，测试图像理解。
5. 测量 RAM、KV Cache、首 Token 延迟和推理速度。

你当前的硬件与软件基础已经足够支持我们开始这个验证工作。

现在先执行第 **2**、**4** 部分的命令，把 `nvcc --version`、`cmake --version` 和 `git describe --tags --always` 的结果发给我。

如果确认工具链没有问题，我们就进入 TensorRT Edge-LLM 的 CMake 配置阶段。届时我会特别关注 TensorRT 10.3 的兼容问题，避免让你为了这个实验破坏已经修复好的 Jetson 系统。

参考：NVIDIA TensorRT Edge-LLM v0.9.0 官方安装说明 ↗。

You said:

```
ksmart@ksmart-desktop:~$ ^C
ksmart@ksmart-desktop:~$ /usr/local/cuda-12.6/bin/nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Wed_Aug_14_10:14:07_PDT_2024
Cuda compilation tools, release 12.6, V12.6.68
Build cuda_12.6.r12.6/compiler.34714021_0
ksmart@ksmart-desktop:~$ echo 'export
PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Wed_Aug_14_10:14:07_PDT_2024
Cuda compilation tools, release 12.6, V12.6.68
Build cuda_12.6.r12.6/compiler.34714021_0
```

ChatGPT said:

很好，**CUDA 12.6** 已经确认正常，`nvcc` 的 **PATH** 问题也解决了。

从你的输出可以确认：

检查项	结果

CUDA Compiler	12.6.68
`nvcc` 执行	正常
PATH 配置	已生效
TensorRT	10.3.0（之前已确认）
Jetson	Orin Nano 8GB，L4T R36.4.7

现在不需要重新安装 CUDA。

下一步我们可以继续准备 TensorRT Edge-LLM，但先不要修改 **TensorRT 10.3** 或升级 **JetPack**。我们需要确认所选源码版本是否真正兼容你当前的系统。

下一步：检查构建工具

先执行这组命令：

Bash

```
echo "===== Build Tools ====="
cmake --version
g++ --version | head -n 1
git --version

echo "===== CUDA ====="
nvcc --version | tail -n 1

echo "===== TensorRT ====="
dpkg-query -W libnvinfer-dev

echo "===== Jetson Model ====="
cat /proc/device-tree/model
```

如果 CMake、G++ 和 Git 都正常，我们再进行源码下载和版本兼容性检查。

还有一个重要提醒：NVIDIA 的 TensorRT Edge-LLM 安装流程将模型量化与导出和**Jetson** 端的 **C++** 推理运行时分成两个部分。模型转换通常需要另一台带 NVIDIA GPU 的 x86 Linux 主机，不能假设仅凭 Jetson 就能完成整个部署流程。

TensorRT Edg...

目前建议的路线是：

CUDA 环境验证完成 → 检查编译工具 → 确认 **Edge-LLM** 兼容版本 → 构建 **Runtime** → 部署量化模型。

你已经完成第一步。现在继续检查构建工具即可。

You said:

nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Wed_Aug_14_10:14:07_PDT_2024
Cuda compilation tools, release 12.6, V12.6.68
Build cuda_12.6.r12.6/compiler.34714021_0
ksmart@ksmart-desktop:~$ ^C
ksmart@ksmart-desktop:~$ echo "===== Build Tools ====="
cmake --version
g++ --version | head -n 1
git --version

echo "===== CUDA ====="
nvcc --version | tail -n 1

echo "===== TensorRT ====="

dpkg-query -W libnvinfer-dev

echo "===== Jetson Model ====="
cat /proc/device-tree/model
===== Build Tools =====
cmake version 3.22.1

CMake suite maintained and supported by Kitware
(kitware.com/cmake).
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
git version 2.34.1
===== CUDA =====
Build cuda_12.6.r12.6/compiler.34714021_0
===== TensorRT =====
libnvinfer-dev 10.3.0.30-1+cuda12.5
===== Jetson Model =====
NVIDIA Jetson Orin Nano Developer Kitksmart@ksmart-desktop:~$
…

Show more

ChatGPT said:

很好，现在你的 Jetson Orin Nano 的开发环境基本确认了。

组件	当前版本	状态
Jetson	Orin Nano Developer Kit	正常识别
Ubuntu	22.04.5 LTS	正常
L4T	R36.4.7	正常
CUDA Toolkit	12.6.68	已验证
TensorRT	10.3.0.30	已安装
CMake	3.22.1	已安装
GCC / G++	11.4.0	已安装
Git	2.34.1	已安装
内存	8GB	后续需控制模型占用
存储	423GB 可用	充足

CUDA、**C++** 编译器和 **Git** 都没有问题。现在唯一需要特别注意的是 **TensorRT Edge-LLM** 的版本兼容性。

尤其是你的 TensorRT 10.3 比较旧。即使某个 Edge-LLM 版本支持 Jetson Orin，也不意味着它一定兼容 TensorRT 10.3。

下一步：开始源码兼容性验证

我重新核对了 NVIDIA 的安装文档，发现它明确列出了 **Jetson Orin + JetPack 6.2+ + CUDA 12.6** 的构建路线。

TensorRT Edg...

因此，现在我们可以开始准备源码，但先不升级任何系统组件。

Step 1 — 下载 TensorRT Edge-LLM

在 Jetson 上执行：

Bash

```
mkdir -p ~/robotics
cd ~/robotics
```

```
git clone --branch v0.9.0 \
  https://github.com/NVIDIA/TensorRT-Edge-LLM.git

cd TensorRT-Edge-LLM

git submodule update --init --recursive

git describe --tags --always
```

如果 Git 提示找不到 `v0.9.0` 标签，先停止，不要自动改用 `main`。我们需要确认具体发布版本。

Step 2 — 检查版本依赖

Bash

```
echo "===== Source Version ====="
git describe --tags --always

echo "===== CMake Requirements ====="
grep -nE 'cmake_minimum_required|TRT_VERSION|CUDAToolkit' CMakeLists.txt

echo "===== TensorRT Header Version ====="
grep -E '#define NV_TENSORRT_(MAJOR|MINOR|PATCH)' \
  /usr/include/aarch64-linux-gnu/NvInferVersion.h
```

这些操作不会修改你的 CUDA 和 TensorRT 安装。

Step 3 — 暂不编译

虽然官方文档列出了兼容的 CUDA 平台，但你的 TensorRT 10.3 是否满足这份源码的全部 API 要求，还没有验证。

等我们确定依赖之后，再执行 CMake 配置和编译，这样可以避免不必要的安装和环境冲突。

现在请先完成 **Step 1** 和 **Step 2**。 如果源码版本与头文件检查都正常，我们就继续构建 TensorRT Edge-LLM Runtime，然后着手部署适合 8GB Jetson 的量化视觉语言模型。

顺便提醒一点：你目前的 15W 电源模式不影响这一步源码检查。等真正进行 VLM 性能测试时，我们再统一功率模式，以便得到有可比性的测试数据。

You said:

```
ksmart@ksmart-desktop:~/robotics/TensorRT-Edge-LLM$ echo "=====
Source Version ====="
git describe --tags --always

echo "===== CMake Requirements ====="
grep -nE 'cmake_minimum_required|TRT_VERSION|CUDAToolkit'
CMakeLists.txt

echo "===== TensorRT Header Version ====="
grep -E '#define NV_TENSORRT_(MAJOR|MINOR|PATCH)' \
  /usr/include/aarch64-linux-gnu/NvInferVersion.h
===== Source Version =====
v0.9.0
===== CMake Requirements =====
16:cmake_minimum_required(VERSION 3.20 FATAL_ERROR)
===== TensorRT Header Version =====
#define NV_TENSORRT_MAJOR 10 //!< TensorRT major version.
#define NV_TENSORRT_MINOR 3 //!< TensorRT minor version.
#define NV_TENSORRT_PATCH 0 //!< TensorRT patch version.
```

ChatGPT said:

很好！现在我们已经确认：

- **TensorRT Edge-LLM v0.9.0**：源码已正确下载。
- **CMake 3.22.1**：满足源码要求的最低 3.20。
- **CUDA 12.6**：已安装且 `nvcc` 正常。
- **TensorRT 10.3**：开发头文件与已安装软件包版本一致。

下一步可以正式尝试 **CMake** 配置（**Configure**），但暂时不编译。

需要强调：你的输出只能证明版本和基本工具正常，尚不能证明 TensorRT 10.3 与 Edge-LLM v0.9.0 的全部 C++ API 兼容。

Step 1 — 配置 TensorRT Edge-LLM

在 Jetson 终端执行：

```bash
cd ~/robotics/TensorRT-Edge-LLM

# 确认工具链文件存在
ls cmake/aarch64_linux_toolchain.cmake

# 创建独立构建目录
mkdir -p build
cd build

# 为 Jetson Orin Nano 配置
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DTRT_PACKAGE_DIR=/usr \
  -DCMAKE_TOOLCHAIN_FILE="$HOME/robotics/TensorRT-Edge-LLM/cmake/aarch64_linux_toolchain.cmake" \
  -DEMBEDDED_TARGET=jetson-orin \
  -DCUDA_CTK_VERSION=12.6
```

这些选项对应 NVIDIA 公布的 Jetson Orin / CUDA 12.6 构建路线。不同源码版本可能还需要其他选项；当前先以你本地 v0.9.0 的 CMake 文件为准。

TensorRT Edg...

Step 2 — 看配置结果

如果成功，末尾一般会出现：

```
-- Configuring done
-- Generating done
-- Build files have been written to: ...
```

如果出现以下错误，含义不同：

错误	可能原因
TensorRT not found	库或头文件路径识别失败
Unknown CMake option	当前源码版本的参数不同
Unsupported TensorRT API	TensorRT 10.3 与源码不兼容
CUDA architecture error	CUDA / GPU 架构配置问题
Missing dependency	需要额外构建依赖

CMake 配置成功，也不代表后续 C++ 编译一定成功。

Step 3 — 编译先不要使用全部核心

Jetson Orin Nano 只有 8GB 共享 RAM。等配置成功后，我们可以先使用：

Bash

```
cmake --build build -j2
```

这里的命令应当从仓库根目录执行。使用 `-j2` 是为了降低并行编译时的内存压力，避免直接使用 `-j$(nproc)` 导致 OOM。

但现在先不要执行 **Step 3**。

完成 Step 1 后，把 CMake 输出末尾约 40 行发给我。我们先确认 TensorRT 10.3 是否通过配置，再决定是否直接编译，或者需要采用兼容版本或隔离构建环境。

我们的目标仍然是：在不破坏现有 **Jetson** 系统的前提下，运行 **TensorRT Edge-LLM**，并最终部署 **Qwen VLM** 为 **Microduck** 提供多模态推理。

You said:

```
ksmart@ksmart-desktop:~/robotics/TensorRT-Edge-LLM$ cd
~/robotics/TensorRT-Edge-LLM

# 确认工具链文件存在
ls cmake/aarch64_linux_toolchain.cmake

# 创建独立构建目录
mkdir -p build
cd build

# 为 Jetson Orin Nano 配置
cmake .. \
  -DCMAKE_BUILD_TYPE=Release \
  -DTRT_PACKAGE_DIR=/usr \
  -DCMAKE_TOOLCHAIN_FILE="$HOME/robotics/TensorRT-Edge-
LLM/cmake/aarch64_linux_toolchain.cmake" \
  -DEMBEDDED_TARGET=jetson-orin \
  -DCUDA_CTK_VERSION=12.6
cmake/aarch64_linux_toolchain.cmake
-- Configurable variable CUDA_CTK_VERSION set to 12.6
-- Configurable variable CUDA_CTK_VERSION set to 12.6
-- The C compiler identification is GNU 11.4.0
-- The CXX compiler identification is GNU 11.4.0
-- The CUDA compiler identification is NVIDIA 12.6.68
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/aarch64-linux-gnu-gcc -
skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/aarch64-linux-gnu-g++ -
skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configurable variable CUDA_CTK_VERSION set to 12.6
-- Configurable variable CUDA_DIR set to
/usr/local/cuda/targets/aarch64-linux
-- NVTX profiling DISABLED (use -
DENABLE_NVTX_PROFILING=ON to enable)
-- Found TensorRT: /usr/lib/aarch64-linux-gnu/libnvinfer.so  found
components: OnnxParser
```

-- FMHA Kernels: Excluding SM architectures: EXCLUDE_SM_80;EXCLUDE_SM_86;EXCLUDE_SM_89;EXCLUDE_SM_100;EXCLUDE_SM_101;EXCLUDE_SM_120;EXCLUDE_SM_121
-- Found Python3: /usr/bin/python3.10 (found version "3.10.12") found components: Interpreter
-- XQA Kernels: generating cubins for SM architectures: 87
-- Configuring done
-- Generating done
-- Build files have been written to: /home/ksmart/robotics/TensorRT-Edge-LLM/build
ksmart@ksmart-desktop:~/robotics/TensorRT-Edge-LLM/build$ cmake --build build -j2
Error: /home/ksmart/robotics/TensorRT-Edge-LLM/build/build is not a directory
…

Show more

ChatGPT said:

很好！**TensorRT Edge-LLM v0.9.0** 的 **CMake** 配置已经成功了。

而且你的输出确认了几个重要信息：

- CUDA 12.6.68：成功识别。
- TensorRT 10.3：成功找到。
- Jetson Orin Nano GPU 架构：SM 87，正确。
- C/C++ 编译器：GCC 11.4.0，正常。
- CMake：`Configuring done`、`Generating done`，均成功。

1. 为什么最后一条命令报错？

原因非常简单：你已经在 `build` 目录里面了。

你的当前路径是：

```
Plain text
/home/ksmart/robotics/TensorRT-Edge-LLM/build
```

但执行了：

```
Bash
cmake --build build -j2
```

CMake 因而寻找：

```
Plain text
/home/ksmart/robotics/TensorRT-Edge-LLM/build/build
```

这个目录不存在，所以报错。

这不是 **CUDA**、**TensorRT** 或编译器错误。

2. 正确的编译命令

你现在直接执行：

Bash

```
cmake --build . -j2
```

其中 `.` 表示当前目录。

或者，无论当前在哪个目录，都可以执行：

Bash

```
cmake --build ~/robotics/TensorRT-Edge-LLM/build -j2
```

两种方式效果相同。

3. 我建议保存编译日志

因为你的 TensorRT 是 10.3，而 Edge-LLM v0.9.0 可能使用较新的 TensorRT API，**CMake** 配置成功尚不能证明源码能完整编译。

建议使用：

Bash

```
cmake --build . -j2 2>&1 | tee build.log
```

这样既可以在终端看到进度，又能把编译日志保存到 `build/build.log`。

如果出现编译错误，可以用：

Bash

```
grep -n -E 'error:|fatal error:|undefined reference|FAILED:' build.log | tail -30
```

来快速定位。

4. 编译时需要注意的事项

你的 Jetson 只有 8GB 共享内存，目前处于 15W 功率模式。

使用 `-j2` 是合理的起点，可以避免同时运行太多编译任务造成内存不足。如果出现 `Killed` 或 `out of memory`，再考虑降低到 `-j1`。

现在不需要调整功率模式，也不需要升级 TensorRT。

目前进展：

- ✅ CUDA 12.6 环境
- ✅ TensorRT 10.3 检测
- ✅ CMake 配置
- 🔄 Edge-LLM 源码编译
- ⚪ Runtime 推理验证
- ⚪ Qwen VLM 部署

现在运行：

Bash

```
cmake --build . -j2 2>&1 | tee build.log
```

如果编译成功，我们就检查生成的可执行文件，并准备第一次 TensorRT Edge-LLM 推理测试。

如果失败，把第一处实际编译错误及其前后约 **15** 行发给我即可。

You said:

我前年买 jetson的时候怎么不知道 又 tensorRT-edge-llM 呢

ChatGPT said:

因为你前年（**2024** 年）购买 **Jetson Orin Nano** 的时候，**TensorRT Edge-LLM** 还没有正式发布。

我查了一下 NVIDIA 的发布记录，时间线很清楚。

NVIDIA 边缘 AI 推理技术的发展

时间	技术	主要定位
2023 年	TensorRT-LLM	面向 NVIDIA GPU 的大模型推理优化
2024 年	JetPack 6 / TensorRT	Jetson 上的 AI、视觉和神经网络推理
2025 年	Jetson Generative AI 生态发展	llama.cpp、Ollama 等本地 LLM 方案
2026 年 1 月 8 日	TensorRT Edge-LLM 正式公开介绍	专门面向汽车、机器人等边缘设备
2026 年 6 月	Edge-LLM v0.8.0	明确验证 Jetson Orin 支持
2026 年 9 月 29 日	Edge-LLM v0.11.0	扩展多模态模型、Python Wheel 和部署能力

NVIDIA 于 2026 年 1 月 8 日正式介绍 TensorRT Edge-LLM，当时主要面向 Jetson Thor 和 DRIVE AGX Thor。随后才逐渐完善对 Jetson Orin 的支持。

NVIDIA Tec… +1

为什么这件事对你的 Jetson 很重要？

你买的 Jetson Orin Nano 并没有变，但这两年 NVIDIA 的软件生态发生了很大变化。

过去使用 Jetson，主要是运行：

- YOLO 等计算机视觉模型。
- TensorRT 优化的 CNN、Transformer。
- CUDA 加速的自定义 AI 程序。
- 一些通过 llama.cpp 等框架运行的小型 LLM。

现在则开始能够使用专门针对嵌入式 GPU 优化的框架，部署 LLM、VLM，以及语音和机器人相关模型。

尤其是 **TensorRT Edge-LLM** 开始直接面向 **Physical AI** 和 **Robotics**，这与你正在研究的 Microduck 非常契合。

不过，我想强调一个区别：

TensorRT 本身并不新，**TensorRT-LLM** 也不是新技术；真正新的是 **TensorRT Edge-LLM** 这条专门针对嵌入式设备的产品线。

它不是给 Jetson 增加了新的硬件算力，而是让现有 GPU 能以更适合边缘设备的方式运行生成式 AI 模型。

而且它不会突破 Orin Nano 8GB 的物理内存上限，INT4 量化、KV Cache 管理与 Runtime 优化仍然非常重要。

所以我觉得你现在重新拿出这台 Jetson，很合适。

2024 年它主要是一台 **Edge AI** 开发板；到了 **2026** 年，我们可以把它作为一个真正的多模态机器人智能节点来研究。

而我们现在正在编译的 v0.9.0，实际上是 **2026** 年 **7** 月 **3** 日才发布的版本。

GitHub

你当年不知道它，完全正常。

You said:

tensorRT是什么？

ChatGPT said:

You said:

那我如果直接 在 jetson上面安装了 Ollama 然后跑个llm的话 他也在运行 tensorRT 并被优化么

ChatGPT said:

不是。**Ollama** 即使在 **Jetson** 上使用 **NVIDIA GPU** 运行 **LLM**，也不代表它使用了 **TensorRT** 优化。

这里有个很重要的区别：

> 使用 **CUDA GPU** 加速 ≠ 使用 **TensorRT** 推理引擎。

Ollama 和 TensorRT Edge-LLM 是两条不同的推理技术路线。

1. Ollama 在 Jetson 上是怎么运行的？

正常情况下，Ollama 使用基于 `llama.cpp` 的推理后端。在支持 CUDA 的环境下，其执行链路大致如下：



Ollama 使用 CUDA 优化的矩阵运算和其他 GPU Kernels，但不会自动调用你系统中已经安装的 **TensorRT 10.3**。

2. 两条路线的对比

	Ollama	**TensorRT Edge-LLM**
推理后端	通常为 llama.cpp / GGML	NVIDIA Edge-LLM Runtime
GPU 加速	CUDA	CUDA + TensorRT 优化
常见模型格式	GGUF	优化后的 TensorRT Engine
INT4 量化	支持	支持
KV Cache	支持	支持
Jetson 上部署	相对简单	较复杂
GPU 性能	较高，依具体模型而定	针对 NVIDIA 硬件优化

TensorRT 依赖	不需要	需要

NVIDIA 也将 Ollama/llama.cpp 与 TensorRT 列为不同的推理后端路线。

NVIDIA Dev… +1

因此，TensorRT Edge-LLM 有机会比 Ollama 更快或更省内存，但不能不经实测就认定一定如此。

3. 如何知道 **Ollama** 有没有真正使用 **GPU**？

你可以在 Jetson 上运行模型后，另开一个终端执行：

```bash
ollama ps
```

查看模型是在 GPU、CPU，还是部分 GPU 上运行。

然后执行：

```bash
sudo tegrastats --interval 1000
```

观察 `GR3D_FREQ`（GPU 利用率）和 `RAM`（内存占用）变化。

如果出现明显 GPU 活动，就说明模型正在利用 GPU 加速，但仍然不是通过 **TensorRT**。

4. 对你的 **Microduck** 项目，我建议两套都试

既然我们已经在 Jetson 上完成 TensorRT Edge-LLM v0.9.0 的 CMake 配置，我建议保留这条技术路线。

同时也可以安装 Ollama，作为性能基线（Baseline）。

最有价值的实验不是证明哪个框架更先进，而是在同一台 **Jetson**、相同模型、相近量化精度、相同输入和上下文长度下，测量：

- 模型实际内存占用，包括 KV Cache。
- TTFT（Time To First Token）。
- Token Generation Speed。
- 图像理解的端到端延迟。
- 长时间运行的稳定性。

这样我们就能判断哪套推理引擎更适合作为 Microduck 的多模态智能服务。

一句话总结：**Ollama** 可以使用 **Jetson** 的 **CUDA GPU** 加速，但不会自动使用 **TensorRT**。**TensorRT Edge-LLM** 是另一套专门优化的推理方案。

而你已经安装的 TensorRT 10.3，只有在某个程序明确调用 TensorRT 时才会参与推理。

You said:

目前的 jetpack 6.2 和 tensorRT 不能往上升个级么

Worked for 42s

ChatGPT said:

可以，而且我查到一个重要更新：截至 **2026** 年 **10** 月，**Jetson Orin Nano** 已经支持 **JetPack 7.2.1**，不必再局限于 **JetPack 6.2**。

同时，你也可以只升级 TensorRT，而不升级整个 JetPack。

不过，这两种升级方式的风险和收益差别很大。考虑到你的 Jetson 之前出现过启动黑屏，而且现在终于恢复正常，我建议先分清楚升级目标。

1. 目前到底能升级到哪些版本？

根据 NVIDIA 官方最新版本资料：

组件	你目前的版本	可选择的版本
Jetson Linux	L4T R36.4.7	R36.5.2 / R39.2.1
JetPack	6.x 环境	6.2.3 / 7.2.1
Ubuntu	22.04	24.04（JetPack 7）
CUDA	12.6	13.2.2（JetPack 7）
TensorRT	10.3	10.7（JetPack 6）或 10.16.2（JetPack 7）
TensorRT Edge-LLM	v0.9.0 源码	可评估更新版本

NVIDIA 官方已经将 JetPack 7.2.1 列为支持 Jetson Orin 系列的版本。它采用 Ubuntu 24.04、CUDA 13.2.2 和 TensorRT 10.16.2。

NVIDIA Dev… +1

需要注意，JetPack 6.2.3 的标准 AI Stack 仍然使用 TensorRT 10.3。升级 JetPack 6 的补丁版本不会自动把 TensorRT 升到 10.7。

NVIDIA Develo…

2. 三条升级路线怎么选？

路线 **A**：保持 **JetPack 6**，升级 **TensorRT 10.3 → 10.7**

不更换 Ubuntu 和 CUDA 主版本，系统变化相对小。

NVIDIA 工程师确认 TensorRT 10.7 GA 是 JetPack 6 可升级的版本。

NVIDIA Develo…

但需要手动处理 TensorRT 软件包及相关依赖，仍然可能影响现有项目。

路线 **B**：**JetPack 6.2.3 + TensorRT 10.3**

保留 Ubuntu 22.04 和 CUDA 12.6，升级 Jetson Linux 到 R36.5.2。

主要收益是安全更新、稳定性修复，而不是显著提升 LLM 推理能力。

NVIDIA Develo…

路线 **C**：升级 **JetPack 7.2.1**

获得较新的 CUDA、TensorRT、Ubuntu 和机器人 AI 软件环境。

这是一轮系统平台迁移，建议备份后采用新介质安装或重新刷机，而不是直接在现有 Ubuntu 22.04 上做普通的 `apt upgrade`。

User Guide +1

3. 对 **TensorRT Edge-LLM**，升级到 **JetPack 7** 有什么实际好处？

这一点很关键。

NVIDIA 的 TensorRT Edge-LLM 支持矩阵中：

- **JetPack 7.2 + Jetson Orin**：Official Support，官方测试支持。
- **JetPack 6.2+ + Jetson Orin**：Compatible，兼容但不是相同的官方验证等级。

TensorRT Edg…

所以如果你的目标是长期研究 Qwen VLM、机器人多模态推理，而不是只试运行一次，JetPack 7 确实更有吸引力。

但是升级不会带来这些变化：

- Orin Nano 的 8GB 共享内存不会增加。
- GPU 计算架构仍然是 Ampere SM87。
- 不会因为 TensorRT 版本较新，就自动支持所有新模型或所有量化格式。

- 不保证 Qwen3-VL 的推理速度或内存一定优于旧环境。

而且 JetPack 7 对你已经安装的 CUDA、ROS 2、Python、TensorRT Engine 和其他原生依赖可能有兼容性影响。

4. 我对你这台 Jetson 的具体建议

我倾向于升级到 **JetPack 7.2.1**，但不是直接覆盖当前系统，而是使用另一张 **microSD** 卡做一次全新安装。

你现在的系统位于：

`/dev/mmcblk0p1`

这是你之前确认的 512GB microSD 系统分区。

因此我们可以采用下面的策略：



我建议这样实施：

1. 先备份当前 microSD 的重要数据和配置。
2. 检查当前 QSPI / UEFI 固件是否支持 JetPack 7.2.1。
3. 使用新的 microSD，按照 NVIDIA 的 Jetson Orin Nano 安装指南部署 JetPack 7。
4. 验证 CUDA、TensorRT、GPU 和网络。
5. 重新部署 TensorRT Edge-LLM，测试 Qwen VLM。
6. 最后再考虑是否迁移原有数据。

NVIDIA 现在也提供 Jetson Orin Nano 的 JetPack 7 ISO 安装途径，可以从 USB 启动安装程序，将系统安装到 microSD 或 NVMe。

User Guide

官方入口：

- JetPack 7.2.1 下载及版本说明 ↗
- Jetson Orin Nano 安装指南 ↗

5. 在真正升级前，我想先确定一件事

你前面运行的：

Bash

```
sudo nvpmodel -q
```

显示：

Plain text

```
NV Power Mode: 15W
0
```

这说明当前是 15W 模式，但我还不知道你的板子是否已经启用了 Orin Nano Super 配置。升级 JetPack 时，值得一并确认这点。

我们可以先运行三个只读检查：

```bash
# 查看可用功率模式
sudo nvpmodel -q --verbose

# 查看当前启动固件
sudo efibootmgr -v

# 查看 NVIDIA 软件源
cat /etc/apt/sources.list.d/nvidia-l4t-apt-source.list
```

总结：你的 **Jetson** 不仅可以升级 **TensorRT**，也可以升级到 **JetPack 7.2.1**。

从 Microduck + 本地多模态 AI 的长期项目角度，我更看好 JetPack 7 路线；但鉴于你刚解决启动故障，最稳妥的是保留当前环境，使用新存储介质完成迁移测试，而不是现在直接执行系统升级命令。

Orin Nano Super 的理论性能提升

原始模式与 Super 模式的官方规格对比



You said:

也就是说这次 重装我并不需要把 SD卡卸下来

ChatGPT said:

对，完全正确！这次升级到 **JetPack 7.2.1**，你不需要把现有的 **512GB microSD** 卡从 **Jetson** 上取下来。

NVIDIA 官方已经支持直接通过 USB 安装盘，把新系统安装到 Jetson 内部的 microSD 卡。

User Guide

整个过程如下：



有三个地方特别注意

1. **USB** 是安装介质，**microSD** 是安装目标。 不要把 ISO 直接写到 microSD。
2. 安装时选择的目标应该是你的 512GB microSD（此前系统中显示为 `/dev/mmcblk0`），但安装器中的设备名称可能不同，应同时核对容量和设备标识。
3. 安装会清除目标 **microSD** 上的数据。 所以先备份 Hermes 配置和尚未提交的代码。

另外，安装程序可能会要求更新 Jetson 的 QSPI/UEFI 固件，按 NVIDIA 官方流程处理即可。

User Guide

你之前用于修复 Jetson 的 Ubuntu Live USB，只要容量至少 16GB，而且里面没有需要保留的数据，就可以重新制作成 JetPack 7.2.1 安装盘。

所以你现在的操作会比过去方便得多：不需要拔 **SD** 卡，也不需要使用另一台 **Ubuntu** 主机连接 **Jetson** 进行刷机。

我们下一步就可以从 NVIDIA 官方 JetPack 下载页面 ↗ 获取 JetPack 7.2.1 ISO，制作启动 U 盘。

You said:

什么是 isaccc

ChatGPT said:

你说的应该是 **NVIDIA Isaac**（艾萨克）。

这是 NVIDIA 专门为机器人和 Physical AI（物理人工智能）开发的一整套技术平台。

如果 **TensorRT** 是让 **AI** 模型高效运行的引擎，那么 **Isaac** 就是帮助开发者设计、仿真、训练和部署智能机器人的工具体系。

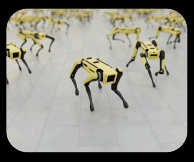
它与你正在研究的 Microduck、强化学习、Jetson Orin Nano 密切相关。

1. Isaac 并不是单一软件

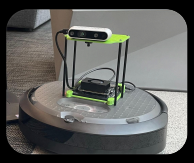
NVIDIA Isaac 包含好几个重要组件：



Isaac Sim — 机器人仿真
在虚拟物理世界中模拟机器人、摄像头、关节、重力、碰撞和环境。
可以先在计算机里测试机器人，再部署到实体设备。



Isaac Lab — 强化学习训练
使用大量并行模拟的机器人，通过 PPO 等算法训练运动 Policy。
它与你刚才学习的 **mjlab** 最接近。



Isaac ROS — 机器人运行时组件
基于 ROS 2，提供 GPU 加速的感知、视觉定位、导航等功能。
这个部分与你的 **Jetson Orin Nano** 最直接相关。



Isaac GR00T — 机器人基础模型
主要面向人形机器人，让模型能够根据感知信息和任务指令生成机器人行为。

这些都是 NVIDIA 官方 Isaac 平台的重要组成部分。

Getting Star… +1

2. Isaac Lab 和 mjlab 有什么区别？

你前面研究 Microduck 时看到的 mjlab 与 Isaac Lab，在定位上非常相似。

	mjlab	Isaac Lab
核心用途	机器人强化学习	机器人强化学习
仿真技术	MuJoCo / MuJoCo Warp	PhysX、Newton 等后端
训练算法	PPO 等	PPO 等
GPU 并行训练	支持	支持

训练产物	Robot Policy	Robot Policy
适合 Microduck	已有相关训练项目	需要自行适配机器人模型

值得注意的是，2026 年发布的 Isaac Lab 3.0 还支持脱离完整 Isaac Sim 独立运行，不再必须启动庞大的图形仿真应用。

Key Difference...

3. 在我们的 **Microduck** 项目里，**Isaac** 处于哪个位置？



这是一种建议的扩展架构，并不是 Microduck 官方已经集成 Isaac ROS 的意思。

4. 你现在应该学 **Isaac** 的哪一部分？

考虑到你打算升级 Jetson，并希望后续进入机器人软件行业，我建议：

- 先学 **Isaac ROS**：了解 ROS 2 的节点、Topic、Service、Action，以及 GPU 加速视觉处理。
- 然后学 **Isaac Lab**：与你目前研究的 mjlab 对照，理解机器人强化学习训练。
- 最后研究 **Isaac Sim**：构建机器人数字孪生，进行更复杂的仿真测试。

另外，NVIDIA 刚在 **2026** 年 **10** 月 **21** 日的 **GTC Berlin** 发布了 Isaac Lab 3.0 和新的 Isaac OS Runtime，说明其机器人开发平台还在快速发展。

NVIDIA Newsr...

官方入口：NVIDIA Isaac Developer Platform ↗。

一句话总结：**TensorRT** 解决“**AI** 模型如何高效运行”，**Isaac** 解决“如何构建、训练并集成机器人智能系统”。而 **Isaac ROS** 是我们最值得在 **Jetson** 上进一步研究的部分。

You said:

iso 下载完了 怎么直接烧录到 usb里面？

Worked for 10s >

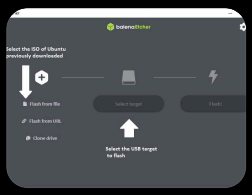
ChatGPT said:

你现在已经下载了 **JetPack 7.2.1 ISO**，可以直接用 Windows 电脑把它制作成启动 U 盘。

我建议使用 **balenaEtcher**，因为这是 NVIDIA 官方 Jetson Orin Nano 安装指南推荐的方法。

User Guide

第一步：下载 balenaEtcher



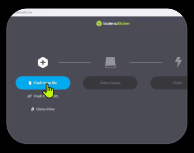
balenaEtcher

免费开源的启动介质写入工具，支持 Windows、Linux、macOS。

下载 balenaEtcher 官方版本 ↗

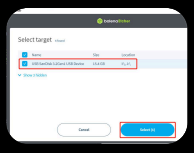
准备一个 至少 **16GB** 的 **USB 3.0 U** 盘。写入会清除 U 盘现有的数据。

第二步：把 ISO 写入 U 盘



1. Flash from file

打开 balenaEtcher，选择你下载的 NVIDIA JetPack 7.2.1 `.iso` 文件。



2. Select target

选择你插入的 USB U 盘，仔细核对容量，避免选错其他磁盘。



3. Flash!

开始写入。Etcher 会将 ISO 写入 U 盘，并验证写入结果。

写入完成后，如果 Windows 提示需要格式化 U 盘，不要格式化。Windows 可能无法识别 Linux 安装介质上的某些分区，这是正常现象。

第三步：在 Jetson 上从 USB 启动

1. 关闭 Jetson 的电源。
2. 保持原来的 **512GB microSD** 卡插在 **Jetson** 内。
3. 插入刚制作好的 JetPack USB 启动盘。
4. 连接显示器和键盘，启动 Jetson。
5. 在 NVIDIA 启动画面出现时按 `Esc`，进入 UEFI 菜单。
6. 选择 **Boot Manager** → **USB** 设备。

启动后，按照安装器提示进行固件更新和系统安装。安装器会让你选择目标存储，选择原来的 512GB microSD 卡，并确认清除原有数据。

User Guide

> ⚠ 特别注意
>
> - U 盘：JetPack 安装介质。
> - 512GB microSD：安装目标。
> - ISO 不能直接复制粘贴到 U 盘，必须通过 Etcher 写入。
> - 原 microSD 上的 Hermes、代码及其他数据将在重装时被清除，先确认已备份。

NVIDIA 官方完整指南：Jetson Orin Nano — JetPack 7.2.1 Quick Start ↗。

我的建议：现在先完成 **Etcher** 的 **Flash** 和验证。 接下来启动 Jetson 时，重点留意安装器里的 **QSPI Firmware Update** 提示，因为这次从 JetPack 6 升级到 7，可能涉及启动固件更新。

You said:

我的U盘没识别出来 是不是因为它是一张 Ubuntu的启动盘？

ChatGPT said:

有可能，而且这种情况很常见。

你之前把这个 U 盘制作成了 Ubuntu Live USB，它可能包含 Linux 的 EFI、ext4 或其他分区。Windows 有时无法正常显示这些分区。

不过要区分两种情况：

- **Windows** 文件资源管理器里看不到 **U** 盘：很可能只是分区格式导致的，不代表 U 盘坏了。
- **balenaEtcher** 的 **Select target** 里也看不到 **U** 盘：需要进一步检查 Windows 是否识别到了物理设备。

第一步：检查 Windows 磁盘管理

在 Windows 电脑上：

1. 按 `Win + X`。
2. 选择 磁盘管理（**Disk Management**）。
3. 找到你的 U 盘，通常显示为“磁盘 1”或“磁盘 2”。
4. 根据容量判断，例如 32GB、64GB。

你可能会看到类似这样的分区布局：



如果磁盘管理能看到这块 U 盘，说明 Windows 至少识别了它的物理存储设备。

第二步：如果 Etcher 仍然无法识别

可以用 Windows 自带的 `diskpart` 清理旧的 Ubuntu 分区。

⚠ 注意：以下操作会清除所选磁盘的所有分区和数据。

一定要通过容量和设备信息确认是 USB，绝不能选到电脑的系统 SSD 或 Jetson 的 microSD。

以管理员身份打开 Windows Terminal，执行：

```powershell
diskpart
list disk
```

找到 U 盘对应的磁盘编号，比如 `Disk 2`。

确认之后：

```
select disk 2
detail disk
```

这里的 `2` 仅是例子，必须替换为你的 U 盘实际编号。检查 `detail disk` 中的容量、设备名称和连接类型。

确认无误后再执行：

```
clean
exit
```

`clean` 会移除分区信息，使 U 盘显示为未分配空间。

然后重新打开 balenaEtcher：

Flash from file → JetPack 7.2.1 ISO → Select target → Flash!

不需要提前格式化成 FAT32，Etcher 会负责写入安装镜像。

我建议你现在先不要执行 `clean`。

先打开 Windows 的 磁盘管理，看看能否找到这张 Ubuntu 启动 U 盘。

你告诉我一个信息就行：是在 **Windows** 文件资源管理器里看不到 **U** 盘，还是 **balenaEtcher** 的 **Select target** 里也看不到？

这两种情况的处理方式不同，我们可以避免不必要的磁盘清理操作。

You said:

我需要找一块新U盘

现在先把 当前SD里面 hermes的环境 尤其是一些 token 和 git PAT 以及 git SSH等等的设置备份下来

我想还是让他传到git上 等下 重装jetson 7之后再完整download下来

ChatGPT said:

这个方案可以，而且趁现在 Jetson 系统还正常、准备重装 JetPack 7，正是做一次完整迁移备份的好时机。

不过我建议稍微调整一下：

使用 **Git** 保存 **Hermes** 的配置、代码和环境重建脚本；使用加密备份保存 **Token**、**Git PAT**、**SSH Private Key** 等敏感信息。

不要直接把整个 `~/.ssh`、`.env` 或 Hermes 的 Token 文件提交到普通 Git 仓库，即使是 Private Repository，也不建议将明文凭据写入 Git 历史。

我们的目标是：重装 JetPack 7 后，只需克隆仓库、解密凭据备份、运行恢复脚本，就能尽可能恢复原来的 Hermes 环境。

一、建议的备份结构



还有一点：JetPack 6 使用 Ubuntu 22.04，JetPack 7 使用 Ubuntu 24.04，因此不应该直接复制旧系统中的 **Python** 虚拟环境、**CUDA** 库或二进制文件。应该保存依赖清单，在新系统中重新安装。

二、Hermes 其实已经有官方备份功能

这比我们自己逐个复制 Hermes 文件更可靠。

官方提供：

- `hermes backup`：备份完整 Hermes 状态，包括配置、认证信息、记忆、Skills 和 Sessions。
- `hermes import`：在新系统中恢复备份。

需要注意：**Hermes** 生成的是普通 **ZIP** 文件，并没有自动加密。 其中可能包含 API Keys 和 OAuth Tokens。

Hermes Agent +1

Step 1 — 先生成完整 Hermes 备份

在目前的 Jetson 上运行：

```bash
mkdir -p ~/jetson-migration

hermes backup -o ~/jetson-migration/hermes-backup.zip

ls -lh ~/jetson-migration/hermes-backup.zip
```

如果 `hermes backup` 不支持，先运行 `hermes --help`，确认你的安装版本是否包含该命令。

Step 2 — 备份 Git 和 SSH 配置

由于你特别提到了 Git PAT 和 SSH Keys，我们还要备份用户级设置。

```bash
cd ~/jetson-migration

tar -czf git-ssh-backup.tar.gz \
  -C "$HOME" \
  --ignore-failed-read \
  .ssh \
  .gitconfig \
  .git-credentials \
  .config/gh

chmod 600 git-ssh-backup.tar.gz
```

这些文件不一定全部存在；如果出现缺失文件提示，要核对归档内容，不能假设所有凭据都已经保存。

Git PAT 也可能由 Git Credential Manager、系统 Keyring 或环境变量管理，未必存在 `.git-credentials` 文件中。SSH Agent 中的临时状态也不能简单通过复制文件恢复。

Step 3 — 对两个备份文件加密

推荐使用 `age`，它适合文件加密，后续恢复也比较方便。

如果当前系统没有 `age`，可以先安装：

```bash
sudo apt install age
```

然后运行：

```bash
cd ~/jetson-migration

age -p -o hermes-backup.zip.age hermes-backup.zip
```

```
age -p -o git-ssh-backup.tar.gz.age git-ssh-backup.tar.gz
```

它会要求你输入加密密码。请使用独立的强密码，并在 **Jetson** 以外的密码管理器中保存。 重装系统后仍然需要这个密码解密。

确认两个 `.age` 文件均已生成后，可以删除临时明文备份：

Bash

```
rm -i hermes-backup.zip git-ssh-backup.tar.gz
```

不要删除整个 `~/jetson-migration` 目录。

三、把加密备份上传到 GitHub

在 GitHub 上新建一个 Private Repository，例如：

`jetson-migration`

然后在 Jetson 上执行：

Bash

```
cd ~/jetson-migration

git init
git branch -M main

cat > .gitignore <<'EOF'
*.zip
*.tar.gz
.env
.env.*
*.pem
*.key
id_*
!*.age
EOF

git add .gitignore \
  hermes-backup.zip.age \
  git-ssh-backup.tar.gz.age

git commit -m "Backup Jetson Hermes and encrypted credentials"
```

然后添加实际创建的 GitHub 私有仓库地址：

Bash

```
git remote add origin git@github.com:YOUR_USERNAME/jetson-migration.git

git push -u origin main
```

这里的 `YOUR_USERNAME` 要换成你自己的 GitHub 用户名。

提交前建议执行：

Bash

```
git status
git diff --cached --name-only
```

确认没有 `.env`、私钥或明文 Token 被加入提交。

非常重要：你至少还应在 Windows 电脑或另一处存储上保留一份 `.age` 文件副本，并验证 GitHub 上确实存在两个加密文件。重装前不能只依赖本地 SD 卡上的备份。

四、重装 JetPack 7 后如何恢复？

新系统安装好之后，先安装 Git 和 age，恢复 GitHub SSH 凭据，然后克隆备份仓库：

```bash
sudo apt update
sudo apt install -y git age openssh-client

git clone git@github.com:YOUR_USERNAME/jetson-migration.git
cd jetson-migration
```

如果 GitHub SSH 尚未恢复，可以先通过 HTTPS 克隆私有仓库，使用新的认证方式。

解密：

```bash
age -d -o hermes-backup.zip hermes-backup.zip.age

age -d -o git-ssh-backup.tar.gz git-ssh-backup.tar.gz.age
```

恢复 SSH / Git 配置前应检查归档内容，然后只解压需要的文件：

```bash
tar -tzf git-ssh-backup.tar.gz
```

确认后：

```bash
tar -xzf git-ssh-backup.tar.gz -C "$HOME"

chmod 700 ~/.ssh
find ~/.ssh -type f -name 'id_*' ! -name '*.pub' -exec chmod 600 {} \;
```

然后按 Hermes 官方安装说明 ↗ 重新安装适用于 JetPack 7 / ARM64 的 Hermes，在新环境中执行：

```bash
hermes import ~/jetson-migration/hermes-backup.zip
```

最后运行 `hermes setup` 检查模型提供商和认证状态。某些 OAuth Token 可能因设备或会话变化而需要重新授权。

五、我建议额外保存一份环境清单

这样以后还可以把整个 Jetson 环境做成可复现部署。

```bash
mkdir -p ~/jetson-migration/environment

dpkg-query -W > ~/jetson-migration/environment/packages.txt
```

```
python3 --version > ~/jetson-migration/environment/python.txt

/usr/local/cuda/bin/nvcc --version \
  > ~/jetson-migration/environment/cuda.txt

git -C ~/robotics/TensorRT-Edge-LLM \
  rev-parse HEAD \
  > ~/jetson-migration/environment/tensorrt-edge-llm-commit.txt
```

这些清单主要用作参考，不建议直接在新 Ubuntu 上逐项重新安装旧系统的所有软件包。

我建议现在先完成 **Hermes** 官方备份和 **Git/SSH** 加密备份，再进行重装。

你目前的任务并不复杂，真正重要的是确保这些凭据、Hermes Sessions 和配置在重装之后还能恢复。

还有一个容易遗漏的点：如果 Hermes 的某些工具使用了独立的 `HERMES_HOME`、系统服务账户或 Docker Volume，默认 `hermes backup` 不一定涵盖那些外部存储位置。正式刷机前应该确认这一点。

只要完成备份、加密、上传和异地验证这四步，我们就有足够把握开始重装。

You said:

你能不能 给我一个 可以粘贴的 完整脚本 我在jetson上直接执行
我现在的 hermes有直接提交和控制 我现有git的能力 我想让他直接就备上去
然后 一会我们装好后 在restore

Worked for 1m 6s

ChatGPT said:

可以。我建议直接做成 **Jetson Migration Backup v1**，一次执行完成备份、加密、Git Commit 和 Push。

这份脚本会保存：

- **Hermes** 完整状态：配置、记忆、Sessions、Skills、认证信息及相关数据。
- **Git** 身份与认证：SSH Keys、Git Config、PAT（如果保存在本地 Git Credentials 中）、GitHub CLI 配置。
- **Jetson** 环境清单：CUDA、TensorRT、已安装软件包、Python 版本。
- 代码仓库清单：记录需要重装后重新 Clone 的项目位置。

Hermes 官方确认 `hermes backup` 支持完整环境迁移，包括认证和会话，但不会备份 Hermes 程序源码、下载的模型及运行时。

GitHub +1

所有敏感文件会先被打包，再通过 **age** 密码加密，Git 仓库中只提交加密后的文件。

你只需要准备一个已经存在的 **GitHub Private Repository**，例如 `jetson-migration`。脚本会让你输入仓库 SSH 地址，并使用 Jetson 已有的 Git SSH 身份推送到独立备份分支，不会覆盖项目的 `main` 分支。

一、完整备份脚本（直接复制到 **Jetson**）

这是一整段 Bash 脚本。复制到 Jetson Terminal 执行即可。过程中需要你输入 GitHub 仓库地址和备份加密密码。

Bash

```
cat > ~/backup-jetson.sh <<'SCRIPT'
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077

STAMP="$(date +%Y%m%d-%H%M%S)"
BRANCH="backup/jetson-$STAMP"
```

```
DEST="$HOME/jetson-migration-$STAMP"
WORK="$(mktemp -d "$HOME/.jetson-backup-tmp.XXXXXXXX")"

cleanup() {
    rm -rf -- "$WORK"
}
trap cleanup EXIT

echo "========================================"
echo "   JETSON MIGRATION BACKUP v1"
echo "========================================"

# 1. Dependencies
```

二、执行后的预期结果

当脚本成功完成，你会看到类似：

Plain text

```
[1/6] Backing up Hermes...
Hermes archive verified.

[2/6] Collecting environment metadata...
[3/6] Collecting Git/SSH configuration...
[4/6] Encrypting backup...
[5/6] Creating Git backup branch...
[6/6] Pushing encrypted backup...

BACKUP PUSH VERIFIED
Branch: backup/jetson-20261025-xxxxxx
```

特别重要：保存加密密码

这个密码不会被提交到 GitHub。如果重装后忘记密码，GitHub 上的加密备份也无法正常恢复。

建议保存在 Windows 电脑的密码管理器里。

三、重装后如何恢复？

这里先给你一个恢复流程预览。等你升级完 JetPack 7，我们可以根据新系统的实际情况一起执行。

Bash

```
# 1. Install utilities
sudo apt update
sudo apt install -y git age

# 2. Clone the backup branch
# Replace YOURNAME and BRANCH with actual values
git clone \
  --branch BRANCH \
  https://github.com/YOURNAME/jetson-migration.git \
  ~/jetson-migration

cd ~/jetson-migration

# 3. Verify and decrypt
sha256sum -c SHA256SUMS

age -d \
  -o jetson-restore.tar.gz \
  jetson-secrets.tar.gz.age

# 4. Extract into a temporary directory
```

之后再恢复 `~/.ssh`、Git 配置，重新安装 Hermes，并执行：

Bash

```
hermes import ~/jetson-restore/hermes.zip
hermes setup
```

恢复前应先停止 Hermes Gateway，避免在运行中的 Hermes 数据库上覆盖状态。

Hermes Agent

四、在真正重装前，还要确认三件事

这个脚本会备份 Hermes 和常见的 Git/SSH 凭据位置，但有几项不属于它的自动备份范围：

- Git 仓库里的未提交修改、未推送的 **Commit** 和 **Untracked Files**：脚本只记录仓库位置，不上传源代码。请让 Hermes 先检查并 Push 需要保留的项目。
- 如果 PAT 保存在 GNOME Keyring、特殊 Credential Helper、环境变量或其他 Secret Store 中，可能需要单独迁移。
- Hermes 的浏览器 Profile、外部 Docker Volume、下载的模型以及其他独立数据目录，不保证包含在官方完整备份中。

另外，加密文件推送后，即使脚本显示成功，仍然建议通过 GitHub 网页验证文件确实存在，并在 Windows 上保留第二份。只有完成这些检查，才能安全擦除 microSD。

我的建议是：先执行这段脚本，等出现 `BACKUP PUSH VERIFIED` 后，把最后几行非敏感输出发给我。 我们再进行最终备份检查，然后安装 JetPack 7。

这套流程也可以保留下来，作为以后 Jetson 开发环境迁移和灾难恢复的基础。